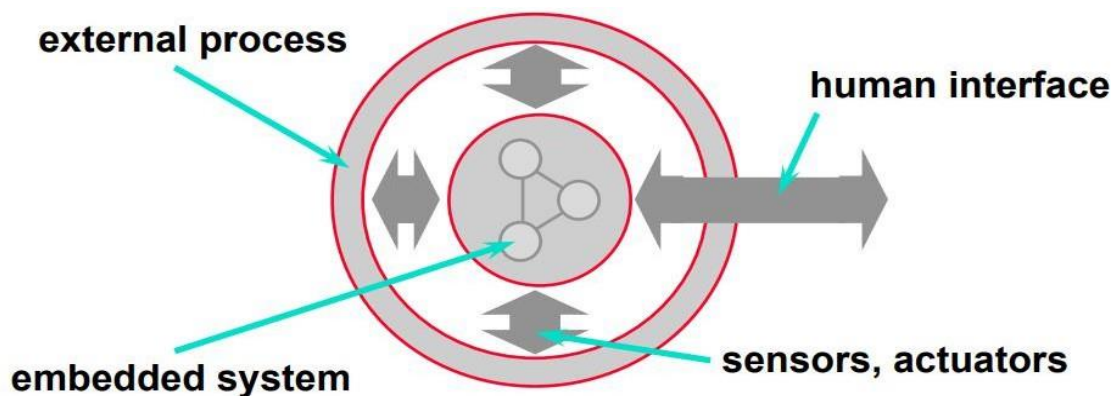


Embedded System

An embedded system is a special-purpose system in which the computer is completely encapsulated by the device it controls. Unlike a general-purpose computer, such as a personal computer, an embedded system performs pre-defined tasks, usually with very specific requirements. Since the system is dedicated to a specific task, design engineers can optimize it, reducing the size and cost of the product. Embedded systems are often mass-produced, so the cost savings may be multiplied by millions of items.

Handheld computers or PDAs are generally considered embedded devices because of the nature of their hardware design, even though they are more expandable in software terms. This line of definition continues to blur as devices expand.

Hence, it is an information processing systems embedded into a larger product. In general it is the core computational part of any automated system shown as below.



Embedded system are founded in verity of electronics devices as below.

1. Consumer Electronics $\Rightarrow$ for example MP3 Audio, digital camera, home electronics, ...
2. Information Systems $\Rightarrow$ for example wireless communication (mobile phone, Wireless LAN, ...), end-user equipment, router, ...
3. Home Appliances $\Rightarrow$ for example micro-oven, Home Security System, washing machines, lighting system...
4. Office Automation $\Rightarrow$ for example fax machines, printer, attendance system, pager.....
5. Business Equipment $\Rightarrow$ for example Cash registers, Card Reader, alarm system, Product Scanner, Automated telling machine
6. Automobiles: for example cruise control, driver assistance system, parking assistance systems, anti-lock brakes.....

Characteristics of Embedded System:

The main characteristics of embedded system are:

Single-Functioned --An embedded system usually executes a specific program reputedly. For example: pager is always pager. A desktop system executes verities of program like spread sheet, word processors

and video games, with new programs added frequently. One case is here the embedded system software be updated to newer versions over a periods of time. For example smart phone software be updated to new versions.

Second case is where several programs are swapped in and out of system due to limitation of size. For example, some missiles run one program while cruise mode, then load a second program for locking into the target.

Tightly Constraint --- All computing system have constraint on design metrics but those on embedded system can be especially tight. A design metric is a measures of implementations feature such as cost, size, power and performance. Embedded system often must cost just a few dollars, must be sized to fit on single size, must perform enough to process the real time data and must consume the minimum power to extend the battery life.

Real Time and Reactive -- Many embedded system most continuously react in many changes in system environments and must compute the certain result in real time without delay. For example a car cruise controller continuously monitors and react to speed and break sensor. It must compute the acceleration and deceleration amount repeatedly with in limited time. A delay computation cause to fail to control the car.

Microprocessors based – It must be microprocessor or microcontroller based.

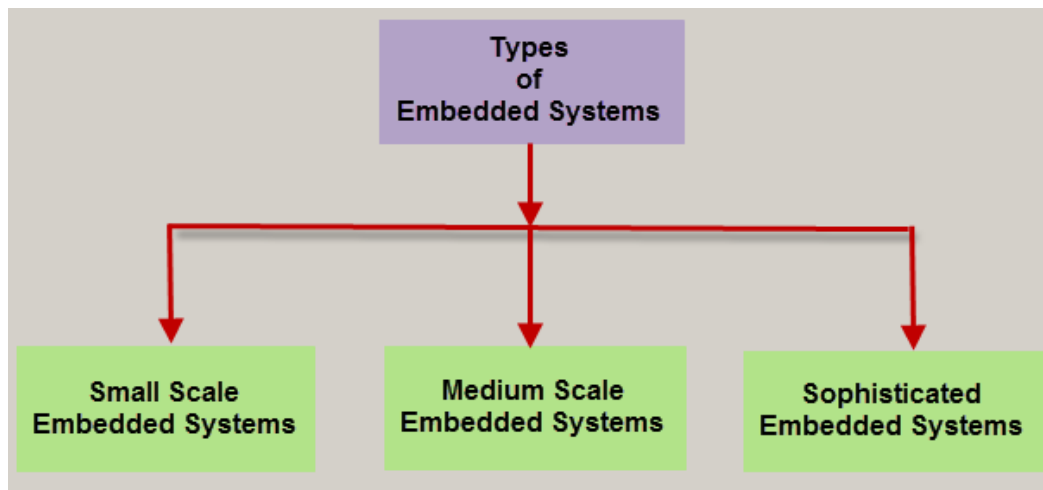
Memory – It must have a memory, as its software usually embeds in ROM. It does not need any secondary memories in the computer.

Connected – It must have connected peripherals to connect input and output devices.

HW-SW systems – Software is used for more features and flexibility. Hardware is used for performance and security.

Classification of Embedded System:

Embedded systems are classified into three:



Small Scale Embedded Systems:

Small scale embedded systems are designed with a single 8 or 16-bit microcontroller which may even be operated with a battery. For developing embedded software for these types of systems, an editor, assembler, (IDE) integrated development environment, and cross assembler are the main programming tools.

Medium Scale Embedded Systems:

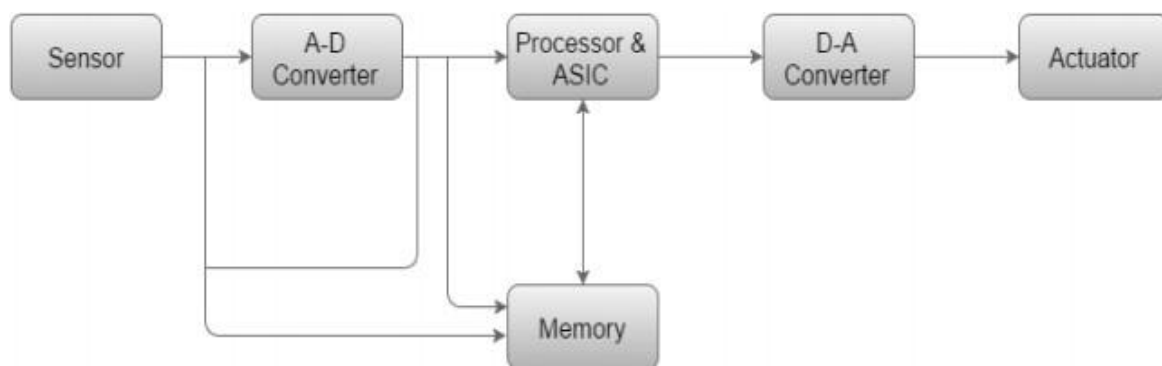
Medium scale embedded systems are designed with a single or few 16 or 32 bit microcontrollers, DSPs or RISCs. These systems have both hardware and software complexities. When developing embedded software for these types of systems, the following programming tools are available. They are C, C++, Visual C++, Java, and RTOS, source code engineering tool, debugger, simulator and integrated development environment.

Sophisticated Embedded Systems:

Sophisticated embedded systems have huge hardware and software complexities and may need PLAs, IPs, ASIPs, scalable processors or configurable processors. They are used for cutting-edge applications that need hardware and software co-design & components which have to combine in the final system.

Basic Structure of an Embedded System:

The following illustration shows the basic structure of an embedded system:



Sensor – It measures the physical quantity and converts it to an electrical signal which can be read by an observer or by any electronic instrument like an A2D converter. A sensor stores the measured quantity to the memory.

A-D Converter – An analog-to-digital converter converts the analog signal sent by the sensor into a digital signal.

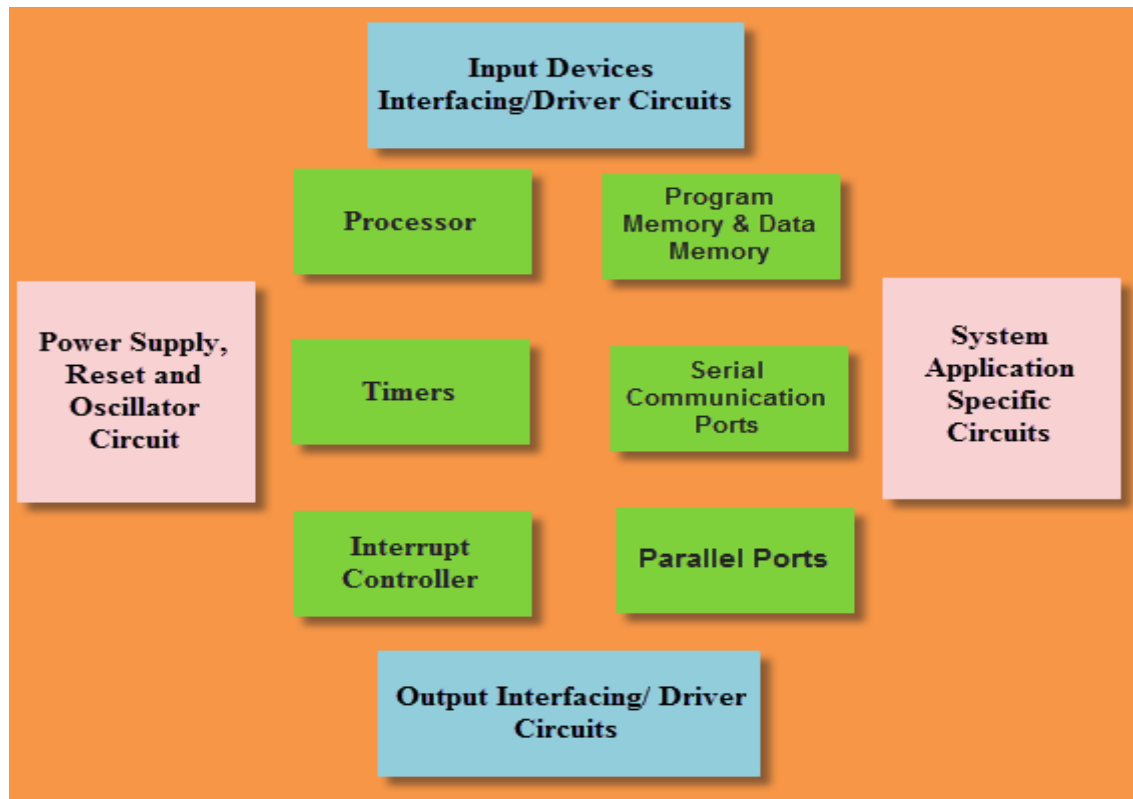
Processor & ASICs – Processors process the data to measure the output and store it to the memory.

D-A Converter – A digital-to-analog converter converts the digital data fed by the processor to analog data.

Actuator – An actuator compares the output given by the D-A Converter to the actual (expected) output stored in it and stores the approved output.

Components of Embedded System:

The embedded systems basics include the components of embedded system hardware, embedded system types and several characteristics. An embedded system has three main components: Embedded system hardware, embedded system software and Operating system.



Embedded system block diagram

Embedded System Hardware:

As with any electronic system, an embedded system requires a hardware platform on which it performs the operation. Embedded system hardware is built with a microprocessor or microcontroller. The embedded system hardware has elements like input output (I/O) interfaces, user interface, memory and the display. Usually, an embedded system consists of:

- Power Supply
- Processor
- Memory
- Timers
- Serial communication ports
- Output/Output circuits
- System application specific circuits

Embedded System Software:

The embedded system software is written to perform a specific function. It is typically written in a high level format and then compiled down to provide code that can be lodged within a non-volatile memory within the hardware. An embedded system software is designed to keep in view of the three limits:

- Availability of system memory
- Availability of processor's speed
- When the system runs continuously, there is a need to limit power dissipation for events like stop, run and wake up.

Real Time Operating System

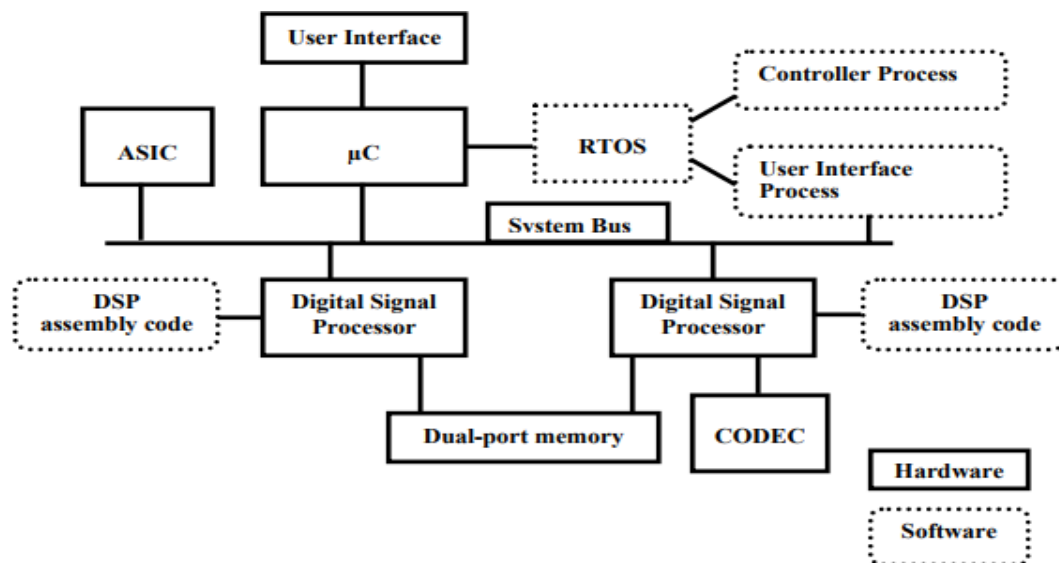
A system is said to be real time, if it is essential to complete its work and deliver its service on time. Real time operating system manages the application software and affords a mechanism to let the processor run. The Real Time operating system is responsible for handling the hardware resources of a computer and host applications which run on the computer.

An RTOS is specially designed to run applications with very precise timing and a high amount of reliability. Especially, this can be important in measurement and industrial automation systems wherein downtime is costly or a program delay could cause a safety hazard.

Architecture of Embedded System:

A Desktop Computer will have more open standards than an Embedded System. This is because of the level of integration in the later. Many of the components of the embedded systems are integrated on to a single chip. This concept is known as **System on Chip (SOC)** design. Thus there are only few subsystems left to be connected.

Analyzing the assembling process of a Desktop let us comparatively assess the possible subsystems of the typical RTES.



Architecture of Embedded System

User Interface: for interacting with users. May consists of keyboard, touch pad etc

ASIC: Application Specific Integrated Circuit: for specific functions like motor control, data modulation etc.

Microcontroller (μC): A family of microprocessors.

Real Time Operating System (RTOS): contains all the software for the system control and user interface
Controller Process: The overall control algorithm for the external process. It also provides timing and control for the various units inside the embedded system.

Digital Signal Processor (DSP) a typical family of microprocessors

DSP assembly code: code for DSP stored in program memory

Dual Ported Memory: Data Memory accessible by two processors at the same time

CODEC: Compressor/Decompressor of the data.

User Interface Process: The part of the RTOS that runs the software for User Interface activities.

Controller Process: The part of the RTOS that runs the software for Timing and Control amongst the various units of the embedded system.

Embedded Systems – Processors:

Processor is the heart of an embedded system. It is the basic unit that takes inputs and produces an output after processing the data. For an embedded system designer, it is necessary to have the knowledge of both microprocessors and microcontrollers.

A processor has two essential units –

- Program Flow Control Unit (CU)
- Execution Unit (EU)

The CU includes a fetch unit for fetching instructions from the memory. The EU has circuits that implement the instructions pertaining to data transfer operation and data conversion from one form to another.

The EU includes the Arithmetic and Logical Unit (ALU) and also the circuits that execute instructions for a program control task such as interrupt, or jump to another set of instructions.

A processor runs the cycles of fetch and executes the instructions in the same sequence as they are fetched from memory.

Types of Processors:

Processors can be of the following categories –

General Purpose Processor (GPP): GPP is used for processing signal from input to output by controlling the operation of system bus, address bus and data bus inside an embedded system. It provides the hardware circuit for memory management i.e. supports the on-chip DMA and Cache. It consists of the common circuitry for computations of arithmetic as well as logical operations used in daily life i.e. it includes the powerful ALU. It uses the large instruction set and uses the pipeline structure for instruction execution to speed up computer. Types of general purpose processor are:

- ✓ Microprocessor
- ✓ Microcontroller
- ✓ Embedded Processor
- ✓ Digital Signal Processor

Microprocessor

A microprocessor is a single VLSI chip having a CPU. In addition, it may also have other units such as caches, floating point processing arithmetic unit, and pipelining units that help in faster processing of instructions.

Earlier generation microprocessors' fetch-and-execute cycle was guided by a clock frequency of order of ~1 MHz. Processors now operate at a clock frequency of 2GHz. Some of the examples are: Intel 8085/8086, 80186, 80286, Motorola 6800, 6809, G3, G4, G5 etc.

Microcontroller

A microcontroller is a single-chip VLSI unit (also called microcomputer) which, although having limited computational capabilities, possesses enhanced input/output capability and a number of on-chip functional units.

CPU	RAM	ROM
I/O Port	Timer	Serial COM Port

Microcontrollers are particularly used in embedded systems for real-time control applications with on-chip program memory and devices. Some of the examples are: Intel 8032, 8051, 8052, AVR ATMEGA 328 etc.

Embedded Processor

An Embedded Processor is a microprocessor that is used in an embedded system. These processors are usually smaller, use a surface mount form factor and consume less power. Embedded processors can be divided into two categories: ordinary microprocessors and microcontrollers. Microcontrollers have more peripherals on the chip. In essence, an embedded processor is a CPU chip used in a system which is not a general-purpose workstation, laptop or desktop computer. For example: ARM 7/9/11, CORETX-M Intel i960 etc.

Digital Signal Processor

A digital signal processor (DSP) is an integrated circuit designed for high-speed data manipulations, and is used in audio, communications, image manipulation, and other data acquisition and data-control applications. For example: PAC, TMS320XX series, Zed-broad etc.

Application Specific System Processor (ASSP): ASSP is application dependent system processor used for processing signal of embedded system. Therefore for different application performing task a unique set of system processors is required.

Application Specific Instruction Processors (ASIPs): ASIP is application dependent instruction processors. It is used for processing the various instruction set inside a combinational circuit of an embedded system.

Design Issues on Embedded System:

The constraints in the embedded systems design are imposed by external as well as internal specifications. Design metrics are introduced to measure the cost function taking into account the technical as well as economic considerations.

Design Metrics on Embedded System:

A Design Metric is a measurable feature of the system's performance, cost, time for implementation and safety etc. Most of these are conflicting requirements i.e. optimizing one shall not optimize the other: e.g. a cheaper processor may have a lousy performance as far as speed and throughput is concerned. Following metrics are generally taken into account while designing embedded systems

NRE cost (nonrecurring engineering cost):

It is one-time cost of designing the system. Once the system is designed, any number of units can be manufactured without incurring any additional design cost; hence the term nonrecurring.

Suppose three technologies are available for use in a particular product. Assume that implementing the product using technology 'A' would result in an NRE cost of \$2,000 and unit cost of \$100, that technology B would have an NRE cost of \$30,000 and unit cost of \$30, and that technology C would have an NRE cost of \$100,000 and unit cost of \$2. Ignoring all other design metrics, like time-to-market, the best technology choice will depend on the number of units we plan to produce.

Unit Cost:

The monetary cost of manufacturing each copy of the system, excluding NRE cost.

Size: The physical space required by the system, often measured in bytes for software, and gates or transistors for hardware.

Performance:

The execution time of the system

Power Consumption:

It is the amount of power consumed by the system, which may determine the lifetime of a battery, or the cooling requirements of the IC, since more power means more heat.

Flexibility:

The ability to change the functionality of the system without incurring heavy NRE cost. Software is typically considered very flexible.

Time-to-prototype:

The time needed to build a working version of the system, which may be bigger or more expensive than the final system implementation, but it can be used to verify the system's usefulness and correctness and to refine the system's functionality.

Time-to-market:

The time required to develop a system to the point that it can be released and sold to customers. The main contributors are design time, manufacturing time, and testing time. This metric has become especially demanding in recent years. Introducing an embedded system to the marketplace early can make a big difference in the system's profitability.

Maintainability:

It is the ability to modify the system after its initial release, especially by designers who did not originally design the system.

Correctness:

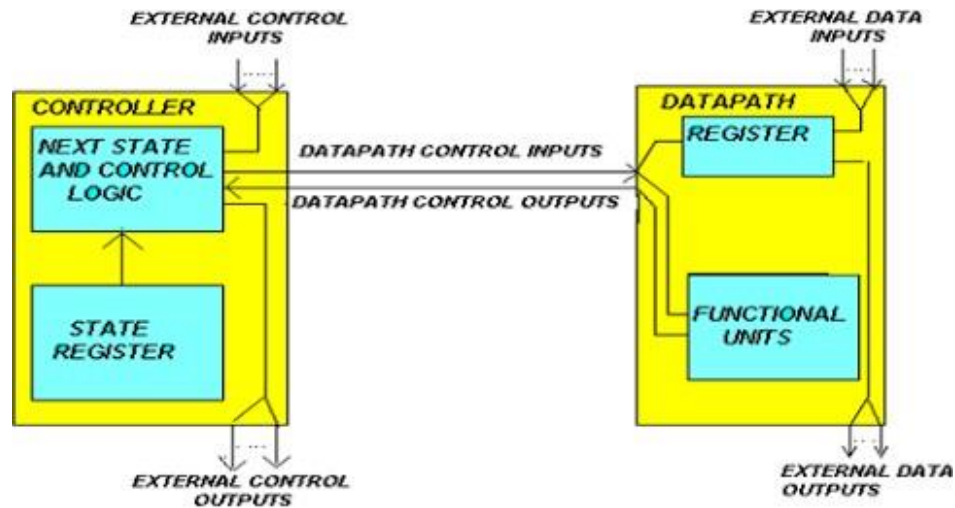
This is the measure of the confidence that we have implemented the system's functionality correctly. We can check the functionality throughout the process of designing the system, and we can insert test circuitry to check that manufacturing was correct.

Single Purpose Processor:-

A single purpose processor is a digital; circuit designed to execute exactly one program. An embedded system designer may obtain several benefits by choosing to use a custom single purpose processor to implement a computation task.

A basic processor consists of a controller and a data path. The data path stores and manipulates a system's data. The data path contains registers units, functional units and connection like wires and multiplexers. The data path can be configured to read data from particular registers feed that data through functional units configured to carry out particular operations like add or shift and store the operation results back in to the particular registers. Controller carries out such configuration of the data path. It sets the data path control inputs, like register load and multiplexer select signals, of the registers units, functional units and connection units to obtain the desired configuration at a particular time.

It monitors external control inputs as well as data path control outputs, known as status signals, coming from functional units, and it sets external control outputs as well. The digital systems design techniques such as combinational and sequential logic design including those of synchronous and asynchronous design can be applied to build a CONTROLLER and a DATA PATH.



Benefits of Custom Single Purpose Processor:

- Performance may be faster, due to fewer clock cycles resulting from a customized data path and due to shorter clock cycles resulting from the simpler controller logic.
- Size may be smaller due to simplest data path and no program memory.
- Power consumption may be less due to more efficient computation.

However, cost could be higher because of high NRE cost. Also time to market may be longer.

Embedded Systems Applications:

Embedded systems have different applications. A few select applications of embedded systems are smart cards, telecommunications, satellites, missiles, digital consumer electronics, computer networking, etc.

- ✓ Embedded Systems in Automobiles
 - Motor Control System
 - Cruise Control System
 - Engine or Body Safety
 - Robotics in Assembly Line
 - Car Entertainment
 - Car multimedia
 - Mobile and E-Com Access
- ✓ Embedded systems in Telecommunications
 - Mobile computing
 - Networking
 - Wireless Communications
- ✓ Embedded Systems in Smart Cards
 - Banking

- Telephone
 - Security Systems
 - ✓ Embedded Systems in Missiles and Satellites
 - Defense
 - Aerospace
 - Communication
 - ✓ Embedded Systems in Computer Networking & Peripherals
 - Networking Systems
 - Image Processing
 - Printers
 - Networks Cards
 - Monitors and Displays
 - ✓ Embedded Systems in Digital Consumer Electronics
 - DVDs
 - Set top Boxes
 - High Definition TVs
 - Digital Camera
-

Chapter- 3

Real Time Operating System (RTOS)

A real-time system is defined as a data processing system in which the time interval required to process and respond to inputs is so small that it controls the environment. The time taken by the system to respond to an input and display of required updated information is termed as the **response time**. So in this method, the response time is very less as compared to online processing.

Real-time systems are used when there are rigid time requirements on the operation of a processor or the flow of data and real-time systems can be used as a control device in a dedicated application. A real-time operating system must have well-defined, fixed time constraints, otherwise the system will fail. For example, scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, air traffic control systems, etc.

There are two types of real-time operating systems.

- **Hard real-time systems**

Hard real-time systems guarantee that critical tasks complete on time. In hard real-time systems, secondary storage is limited or missing and the data is stored in ROM. In these systems, virtual memory is almost never found.

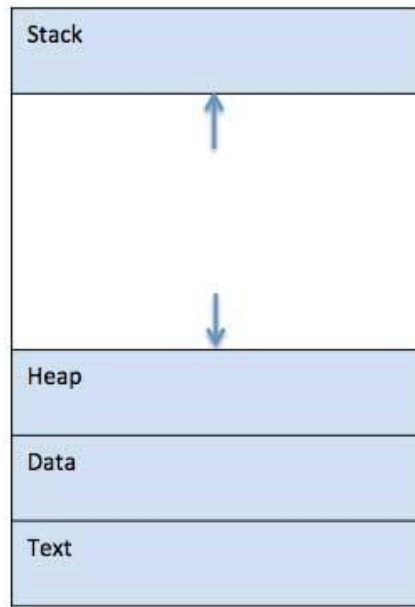
- **Soft real-time systems**

Soft real-time systems are less restrictive. A critical real-time task gets priority over other tasks and retains the priority until it completes. Soft real-time systems have limited utility than hard real-time systems. For example, multimedia, virtual reality, Advanced Scientific Projects like undersea exploration and planetary rovers, etc.

Definition of Process, Task and Thread

Process:

- A process is basically a program in execution. The execution of a process must progress in a sequential fashion.
 - A process is defined as an entity which represents the basic unit of work to be implemented in the system.
 - To put it in simple terms, we write our computer programs in a text file and when we execute this program, it becomes a process which performs all the tasks mentioned in the program.
 - Every process has its own address space.
 - When a program is loaded into the memory and it becomes a process, it can be divided into four sections — stack, heap, text and data. The following image shows a simplified layout of a process inside main memory –
-



- **Stack:** The process Stack contains the temporary data such as method/function parameters, return address and local variables.
- **Heap:** This is dynamically allocated memory to a process during its run time.
- **Text:** This includes the current activity represented by the value of Program Counter and the contents of the processor's registers.
- **Data:** This section contains the global and static variables.

Task:

- A job is a unit of work that is scheduled and executed by a system
 - E.g. computation of a control-law, computation of an FFT on sensor data, transmission of a data packet, retrieval of a file
- A task is a set of related jobs which jointly provide some function
 - E.g. the set of jobs that constitute the “maintain constant altitude” task, keeping an airplane flying at a constant altitude
- The embedded system uses the three types of task as:
 - Periodic Task
 - Aperiodic Task
 - Sporadic Task

Periodic Task:



- The release time of job is known before its event triggering and repeated over the fixed interval of time called as period is known as periodic task and corresponding workload model is called as Periodic task model.
- A periodic task T_i be defined as $T_i = (\phi_i, p_i, e_i, D_i)$ where T_i refers a periodic task with phase ϕ_i , period p_i , execution time e_i , and relative deadline D_i .
- Default phase of T_i is $\phi_i = 0$, default relative deadline is the period $D_i = p_i$.
- Omit elements of the tuple that have default values.

Example:

i) $T_1 = (1, 10, 3, 6) \Rightarrow \phi_1 = 1, p_1 = 10, e_1 = 3, D_1 = 6$

$J_{1,1}$ released at 1, deadline 7 and $J_{1,2}$ released at 11, deadline 17.

ii) $T_2 = (10, 3, 6) \Rightarrow \phi_2 = 0, p_2 = 10, e_2 = 3, D_2 = 6$

$J_{2,1}$ released at 0, deadline 6 and $J_{2,2}$ released at 10 and so on.... deadline 16

iii) $T_3 = (10, 3) \Rightarrow \phi_3 = 0, p_3 = 10, e_3 = 3, D_3 = 10.$

$J_{3,1}$ released at 0, deadline 10 and $J_{3,2}$ released at 10, deadline 20.

Sporadic and Aperiodic Task:

Most embedded systems have to respond to external events which occur randomly. When such an event occurs the system executes a set of jobs in response. The release times of those jobs are not known until the event triggering them occurs. These jobs are called sporadic jobs or aperiodic jobs because they are released at random times.

If the tasks containing jobs that are released at random time instants and have hard deadlines then they are called sporadic task. Sporadic tasks are treated as hard real-time tasks. To ensure that their deadlines are met is the primary concern whereas minimizing their response times is of secondary importance. For example,

- An autopilot is required to respond to a pilot's command to disengage the autopilot and switch to manual control within a specified time.
- A fault tolerant system may be required to detect a fault and recover from it in time to prevent disaster

When the task or job have no any deadlines or soft deadline then it is called aperiodic task or job. For example,

- An operator adjusts the sensitivity of a radar system. The radar must continue to operate and in the near future change its sensitivity.

Threads

- A thread is a simple program that thinks it has the CPU all to itself. The design process for a real-time application involves splitting the work to be done into threads which are responsible for a portion of the problem. Each thread is assigned a priority, its own set of CPU registers and its own stack area.
- Each thread is typically an infinite loop that can be in one of four states: READY, RUNNING, WAITING or INTERRUPTED.

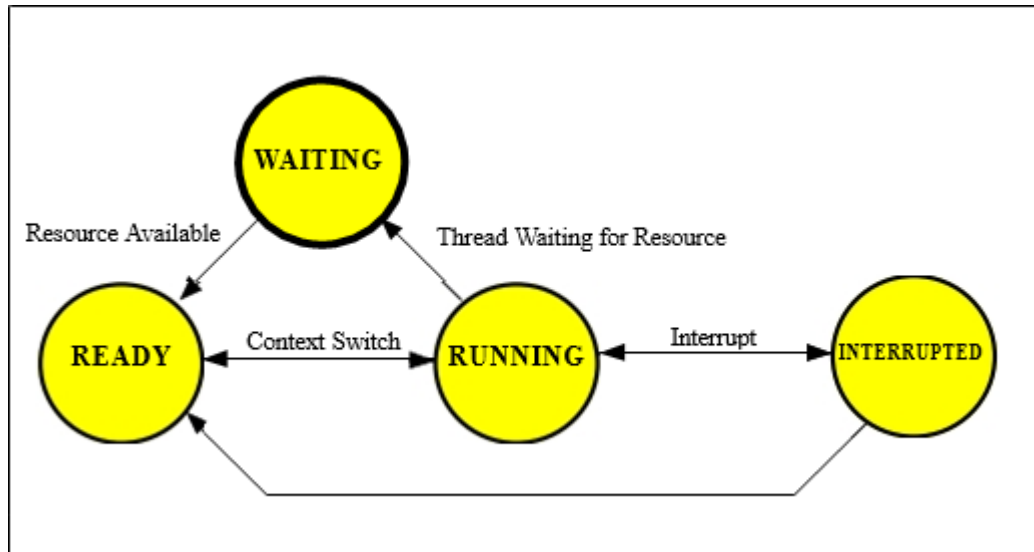
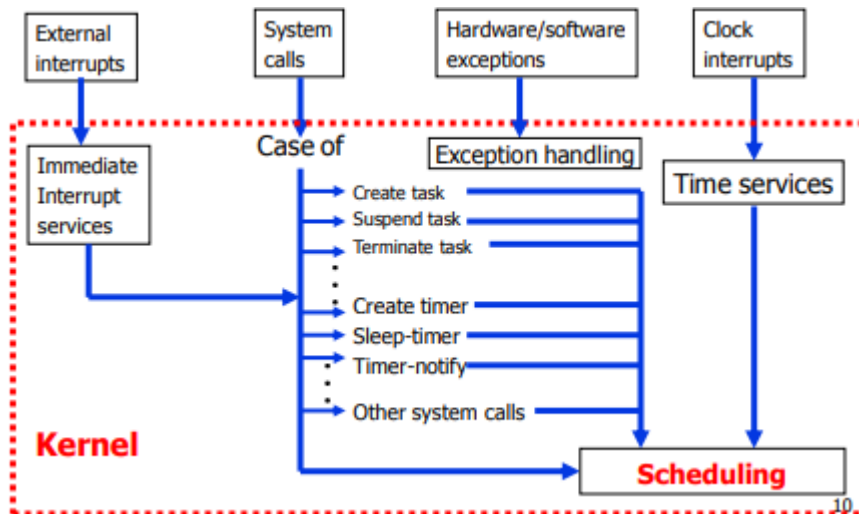


Figure – Thread states

- A thread is **READY** when it can execute but its priority is less than the current running thread.
 - A thread is **RUNNING** when it has control of the CPU.
 - A thread is **WAITING** when the thread suspends itself until a certain amount of time has elapsed, or when it requires the occurrence of an event: waiting for an I/O operation to complete, a shared resource to be available, a timing pulse to occur etc.
 - Finally, a thread is **INTERRUPTED** when an interrupt occurred and the CPU is in the process of servicing the interrupt.

Real-Time Kernel Concepts:

- In most cases the real time os is an operating system kernel.
 - An embedded system is designed for a single purpose so the user shell and file/disk access features are unnecessary.
 - The kernel is the part of an OS that is responsible for the management of threads (i.e., managing the CPU's time) and for communication between threads. The fundamental service provided by the kernel is context switching.
 - RTOS Kernel has following functions:
 - Time management
 - Task management
 - Interrupt handling
 - Memory management
 - Exception handling
 - Task synchronization
 - Task scheduling
-



Time Management

A high resolution hardware timer is programmed to interrupt the processor at fixed rate called as Time interrupt. Each time interrupt is called a system tick (time resolution).

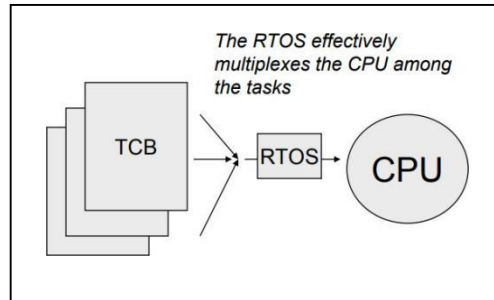
- Normally, the tick can vary in microseconds (depend on hardware)
- The tick may be selected by the user
- All time parameters for tasks should be the multiple of the tick
- Note: the tick may be chosen according to the given task parameters
- System time = 32 bits then
 - One tick = 1ms: your system can run 50 days
 - One tick = 20ms: your system can run 1000 days = 2.5 years
 - One tick = 50ms: your system can run 2500 days = 7 years

The time interrupt routine is used to serve the time interrupt and is a part of the RTOS kernel. Following operations are performed to serve the time interrupt by service routine.

- Save the context of the task in execution v
 - Increment the system time by 1, if current time > system lifetime, generate a timing error
 - Update timers (reduce each counter by 1)
 - Activation of periodic tasks in idling state
 - Schedule again - call the scheduler
 - Other functions e.g. v
 - (Remove all tasks terminated – de-allocate data structures e.g TCBs)
 - (Check if any deadline misses for hard tasks, monitoring)
 - load context for the first task in ready queue
- load context for the first task in ready queue

States of a Task in a system

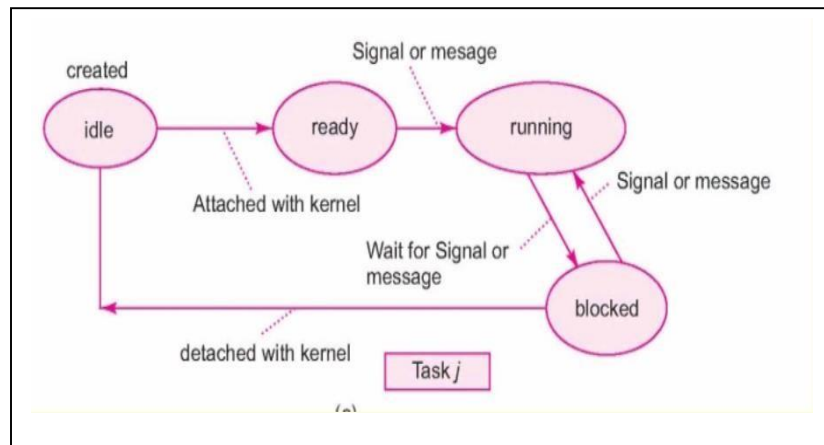
- A task is the combination of code, data and states (context) .Task State is stored in a Task Control Block (TCB) when the task is not running on the processor and the RTOS or kernel selects the task available on the different state as per need for operation.



The number of states required to process the task is deepens on the type and complexity of RTOS. The states of RTOS task are

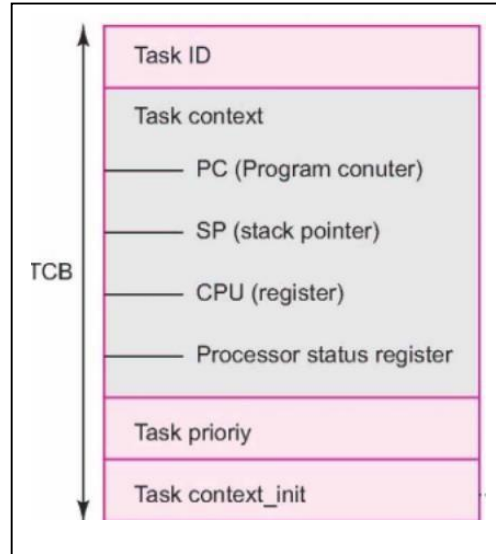
- Idle state
- Ready State
- Running state
- Blocked (waiting) state
- Deleted state

The finite state machine for task state is:



- **Idle (Created) State:** The task has been created and memory allotted to its structure. However, it is not ready and is not schedulable by kernel.
- **Ready (Active) State:** The created task is ready and is schedulable by the kernel but not running at present as another higher priority task is scheduled to run and gets the system resources at this instance.
- **Running state:** Executing the codes and getting the system resources at this instance. It will run till it needs some IPC (input) or wait for an event or till it gets preempted by another higher priority task than this one.
- **Blocked (waiting) state:** Execution of task codes suspends after saving the needed parameters into its context. It needs some IPC (input) or it needs to wait for an event or wait for higher priority task to block to enable running after blocking.
- **Deleted (finished) state:** The created task has memory de allotted to its structure i.e. task be deleted such that It frees the memory.

Task Control Block



- A data structure having the information using which the OS controls the process state.
- Task Information at the TCB are:

TaskID: The unique identifier use to define a task. For example, in case of 8-bit ID, a number between 0 and 255 be used to define TaskID.

Task Context: It includes the current status of program counter, stack pointer, status of CPU register and Status Register.

Task priority: It stores the priority level of parent as well as child task available in Task List. The priority is a number used as the identifier.

Task Context_init: it is a pointer to the processor memory that stores following information.

- Allocated program memory address blocks in physical memory and in secondary (virtual) memory for the tasks-codes.
- Allocated task-specific data address blocks.
- Allocated task-stack addresses for the functions called during running of the process.
- Allocated addresses of CPU register-save area as a task context represents by CPU registers, which include the program counter and stack pointer.

Context Switch

When the multithreading kernel decides to run a different thread, it simply saves the current thread's context (CPU registers) in the current thread's context storage area (the thread control block, or TCB). Once this operation is performed, the new thread's context is restored from its TCB and the CPU resumes execution of the new thread's code. This process is called a context switch. Context switching adds overhead to the application.

Task Management:

The task management operation defines the following operations:

- Creation of new task with TCB.
- Task termination: remove the TCB

- Change Priority: modify the TCB
- State-inquiry: read the TCB



The major challenges for Task Management in RTOS kernel are:

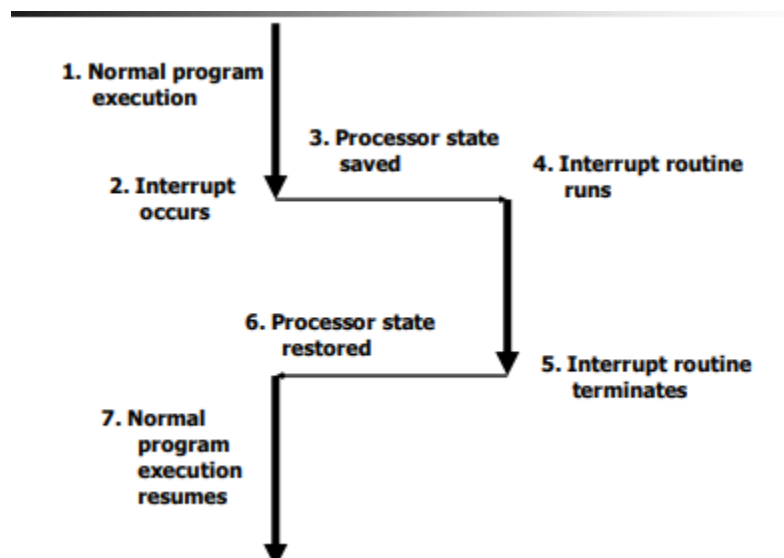
- Creating an RT task, it has to get the memory without delay: this is difficult because memory has to be allocated and a lot of data structures, code segment must be copied/initialized.
- Changing run-time priorities is dangerous: it may change the run-time behavior and predictability of the whole system.

Interrupt Handling:

An interrupt is a hardware mechanism used to inform the CPU that an asynchronous event has occurred. When an interrupt is recognized, the CPU saves all of its context (i.e., registers) and jumps to a special subroutine called an Interrupt Service Routine, or ISR. The ISR processes the event, and upon completion of the ISR, the program returns to:

- the background for a foreground / background system,
- the interrupted thread for a non-preemptive kernel, or
- The highest priority thread ready to run for a preemptive kernel.

Interrupts allow a microprocessor to process events when they occur. This prevents the microprocessor from continuously polling an event to see if it has occurred. Microprocessors allow interrupts to be ignored and recognized through the use of two special instructions: disable interrupts and enable interrupts, respectively.



The interrupt handlers handle the interrupt generated by external devices as below:

- The current context of the task is saved on stack.
- Block the task and branches the program control to beginning address of ISR and executes the ISR to serve the interrupt.
- Terminates from interrupt routine and read the context of the blocked task.

In a real-time environment, interrupts should be disabled as little as possible. Disabling interrupts affects interrupt latency and may cause interrupts to be missed. Processors generally allow interrupts to be nested. This means that while servicing an interrupt, the processor will recognize and service other (more important) interrupts, as shown in Figure below.

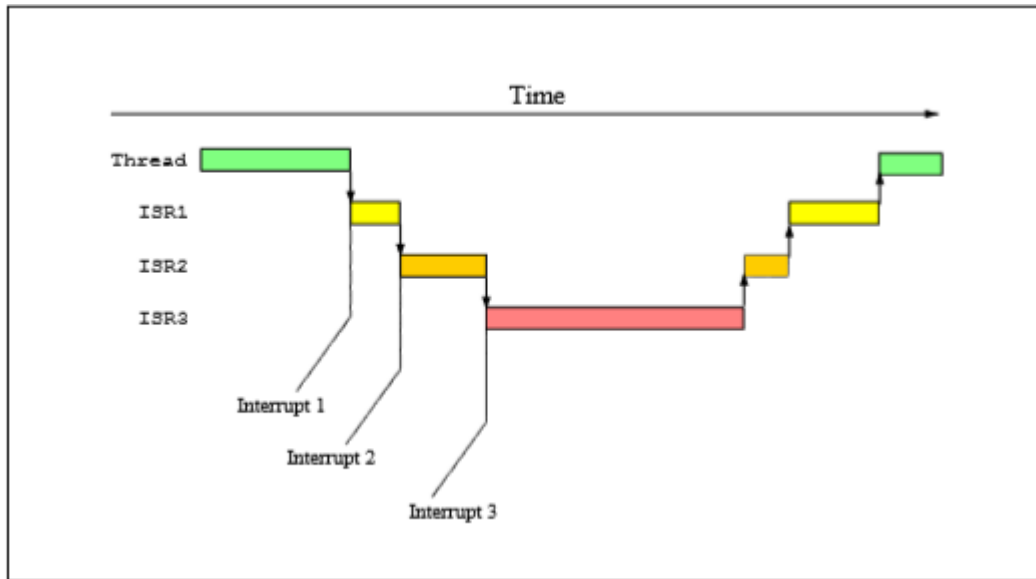


Figure – Interrupt nesting

Interrupt Latency

Probably the most important specification of a real-time kernel is the amount of time interrupts are disabled. All real-time systems disable interrupts to manipulate critical sections of code and renewable interrupts when the critical section has executed. The longer interrupts are disabled, the higher the interrupt latency. Interrupt latency is given by

Interrupt latency = Maximum amount of time interrupts are disabled + Time to start executing the first instruction in the ISR

Interrupt Response

Interrupt response is defined as the time between the reception of the interrupt and the start of the user code that handles the interrupt. The interrupt response time accounts for all the overhead involved in handling an interrupt.

For a foreground / background system, the user ISR code is executed immediately. The response time is given by

Interrupt recovery time = Time to execute the return from interrupt instruction

Interrupt Recovery

Interrupt recovery is defined as the time required for the processor to return to the interrupted code. Interrupt recovery in a foreground / background system simply involves restoring the processor's context and returning to the interrupted thread. Interrupt recovery is given by:

Interrupt recovery time = Time to execute the return from interrupt instruction

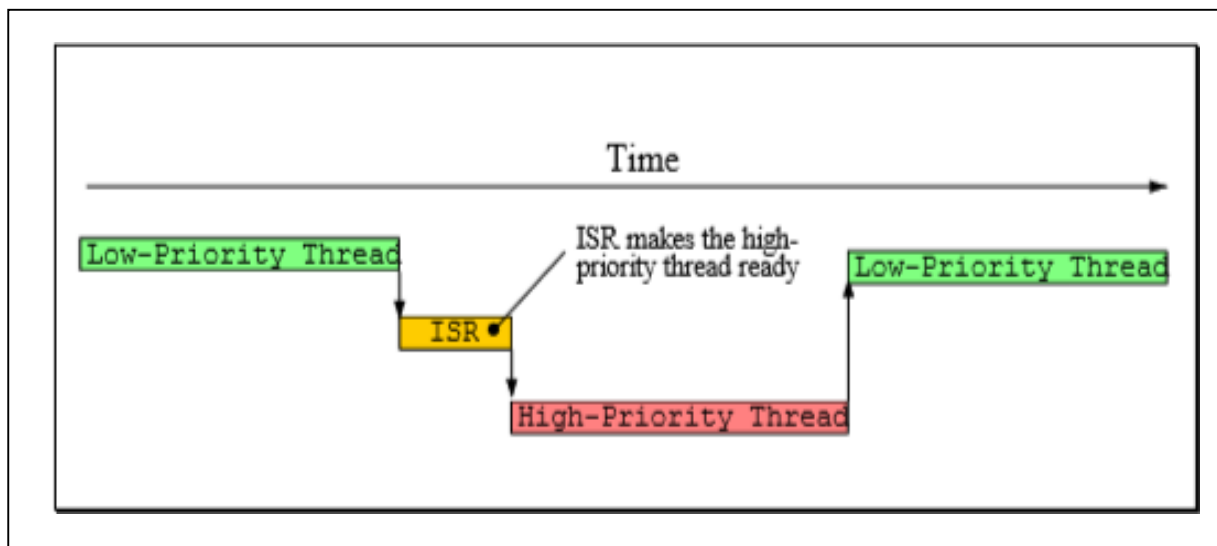
ISR Processing Time

Although ISRs should be as short as possible, there are no absolute limits on the amount of time for an ISR. One cannot say that an ISR must always be less than 100 ms, 500 ms, or 1 ms. If the ISR code is the most

important code that needs to run at any given time, it could be as long as it needs to be. In most cases, however, the ISR should recognize the interrupt, obtain data or a status from the interrupting device, and signal a thread to perform the actual processing.

Scheduler

The scheduler is the part of the kernel responsible for determining which thread will run next. Most real-time kernels are priority based. Each thread is assigned a priority based on its importance. Establishing the priority for each thread is application specific. In a priority-based kernel, control of the CPU will always be given to the highest priority thread ready to run. In a preemptive kernel, when a thread makes a higher priority thread ready to run, the current thread is pre-empted (suspended) and the higher priority thread is immediately given control of the CPU. If an interrupt service routine (ISR) makes a higher priority thread ready, then when the ISR is completed the interrupted thread is suspended and the new higher priority thread is resumed.



With a preemptive kernel, execution of the highest priority thread is deterministic; you can determine when the highest priority thread will get control of the CPU.

Application code using a preemptive kernel should not use non-reentrant functions, unless exclusive access to these functions is ensured through the use of mutual exclusion semaphores, because both a low- and a high-priority thread can use a common function. Corruption of data may occur if the higher priority thread preempts a lower priority thread that is using the function.

To summarize, a preemptive kernel always executes the highest priority thread that is ready to run. An interrupt preempts a thread. Upon completion of an ISR, the kernel resumes execution to the highest priority thread ready to run (not the interrupted thread). Thread-level response is optimum and deterministic.

Reentrancy

A reentrant function can be used by more than one thread without fear of data corruption. A reentrant function can be interrupted at any time and resumed at a later time without loss of data. Reentrant functions either use local variables (i.e., CPU registers or variables on the stack) or protect data when global variables are used. An example of a reentrant function is shown below:

```
char* strcpy(char* dst, const char* src)
{
    char* ptr = dst;
    while (*dst++ = *src++);
    return ptr;
}
```

Since copies of the arguments to `strcpy()` are placed on the thread's stack, and the local variable is created on the thread's stack, `strcpy()` can be invoked by multiple threads without fear that the threads will corrupt each other's pointers.

An example of a non-reentrant function is shown below:

```
static int Temp;

void swap(int* x, int* y)
{
    Temp = *x;
    *x = *y;
    *y = Temp;
}
```

`Swap ()` is a simple function that swaps the contents of its two arguments. Since `Temp` is a global variable, if the `swap ()` function gets preempted after the first line by a higher priority thread which also uses the `swap ()` function, then when the low priority thread resumes it will use the `Temp` value that was used by the high priority thread.

We can make `swap ()` reentrant with one of the following techniques:

- Declare `Temp` local to `swap ()`.
- Disable interrupts before the operation and enable them afterwards.
- Use a semaphore.

Thread Priority

A priority is assigned to each thread. The more important the thread, the higher the priority given to it.

- **Static Priorities**

Thread priorities are said to be static when the priority of each thread does not change during the application's execution. Each thread is thus given a fixed priority at compile time. All the threads and their timing constraints are known at compile time in a system where priorities are static

- **Dynamic Priorities**

Thread priorities are said to be dynamic if the priority of threads can be changed during the application's execution; each thread can change its priority at run time. This is a desirable feature to have in a real-time kernel to avoid priority inversions.

- **Priority Inversions**

Priority inversion is a problem in real-time systems and occurs mostly when you use a real-time kernel. Priority inversion is any situation in which a low priority thread holds a resource while a

higher priority thread is ready to use it. In this situation the low priority thread prevents the high priority thread from executing until it releases the resource.

To avoid priority inversion a multithreading kernel should change the priority of a thread automatically to help prevent priority inversions. This is called priority inheritance.

Mutual Exclusion

The easiest way for threads to communicate with each other is through shared data structures. This is especially easy when all threads exist in a single address space and can reference global variables, pointers, buffers, linked lists, FIFOs, etc. Although sharing data simplifies the exchange of information, we must ensure that each thread has exclusive access to the data to avoid contention and data corruption. The most common methods of obtaining exclusive access to shared resources are:

- disabling interrupts,
- performing test-and-set operations,
- disabling scheduling, and
- Using semaphores.

Semaphores

The semaphore was invented by Edgser Dijkstra in the mid-1960s. It is a protocol mechanism offered by most multithreading kernels. Semaphores are used to:

- control access to a shared resource (mutual exclusion),
- signal the occurrence of an event, and
- Allow two threads to synchronize their activities.

A semaphore is a key that code acquires in order to continue execution. If the semaphore is already in use, the requesting thread is suspended until the semaphore is released by its current owner. In other words, the requesting thread says: "Give me the key. If someone else is using it, I am willing to wait for it!" There are two types of semaphores: binary semaphores and counting semaphores. As its name implies, a binary semaphore can only take two values: 0 or 1. A counting semaphore allows values between 0 and 255, 65535, or 4294967295, depending on whether the semaphore mechanism is implemented using 8, 16, or 32 bits, respectively. The actual size depends on the kernel used. Along with the semaphore's value, the kernel also needs to keep track of threads waiting for the semaphore's availability.

Generally, only three operations can be performed on a semaphore: Create (), Wait (), and Signal (). The initial value of the semaphore must be provided when the semaphore is initialized. The waiting list of threads is always initially empty.

A thread desiring the semaphore will perform a Wait () operation. If the semaphore is available (the semaphore value is greater than 0), the semaphore value is decremented and the thread continues execution. If the semaphore's value is 0, the thread performing a Wait () on the semaphore is placed in a waiting list. Most kernels allow you to specify a timeout; if the semaphore is not available within a certain amount of time, the requesting thread is made ready to run and an error code (indicating that a timeout has occurred) is returned to the caller.

A thread releases a semaphore by performing a Signal () operation. If no thread is waiting for the semaphore, the semaphore value is simply incremented. If any thread is waiting for the semaphore, however, one of the threads is made ready to run and the semaphore value is not incremented; the key is given to one of the threads waiting for it. Depending on the kernel, the thread that receives the semaphore is either:

- The highest priority thread waiting for the semaphore, or
- The first thread that requested the semaphore (First In First Out).

Some kernels have an option that allows you to choose either method when the semaphore is initialized. For the first option, if the readied thread has a higher priority than the current thread (the thread releasing the semaphore), a context switch occurs (with a preemptive kernel) and the higher priority thread resumes execution; the current thread is suspended until it again becomes the highest priority thread ready to run.

Following listing shows how you can share data using a semaphore. Any thread needing access to the same shared data calls `OS_SemaphoreWait()`, and when the thread is done with the data, the thread calls `OS_SemaphoreSignal()`. Both of these functions are described later. You should note that a semaphore is an object that needs to be initialized before it is used; for mutual exclusion, a semaphore is initialized to a value of 1. Using a semaphore to access shared data doesn't affect interrupt latency. If an ISR or the current thread makes a higher priority thread ready to run while accessing shared data, the higher priority thread executes immediately.

```
OS_ECB* SharedDataSemaphore;

void Function(void)
{
    OS_ERROR error;

    error = OS_SemaphoreWait(SharedDataSemaphore, 0);
    .
    .    // You can access shared data in here
    .    // (interrupts are recognized)
    .
    error = OS_SemaphoreSignal(SharedDataSemaphore);
}
```

Semaphores are especially useful when threads share I/O devices. Imagine what would happen if two threads were allowed to send characters to a printer at the same time. The printer would contain interleaved data from each thread. For instance, the printout from Thread 1 printing **"I am Thread 1!"** and Thread 2 printing **"I am Thread 2!"** could result in:

"I Ia amm T Threahread d1 !2!"

In this case, use a semaphore and initialize it to 1 (i.e., a binary semaphore). The rule is simple: to access the printer each thread first must obtain the resource's semaphore.

In this case, use a semaphore and initialize it to 1 (i.e., a binary semaphore). The rule is simple: to access the printer each thread first must obtain the resource's semaphore.

Figure below shows threads competing for a semaphore to gain exclusive access to the printer. Note that the semaphore is represented symbolically by a key, indicating that each thread must obtain this key to use the printer.



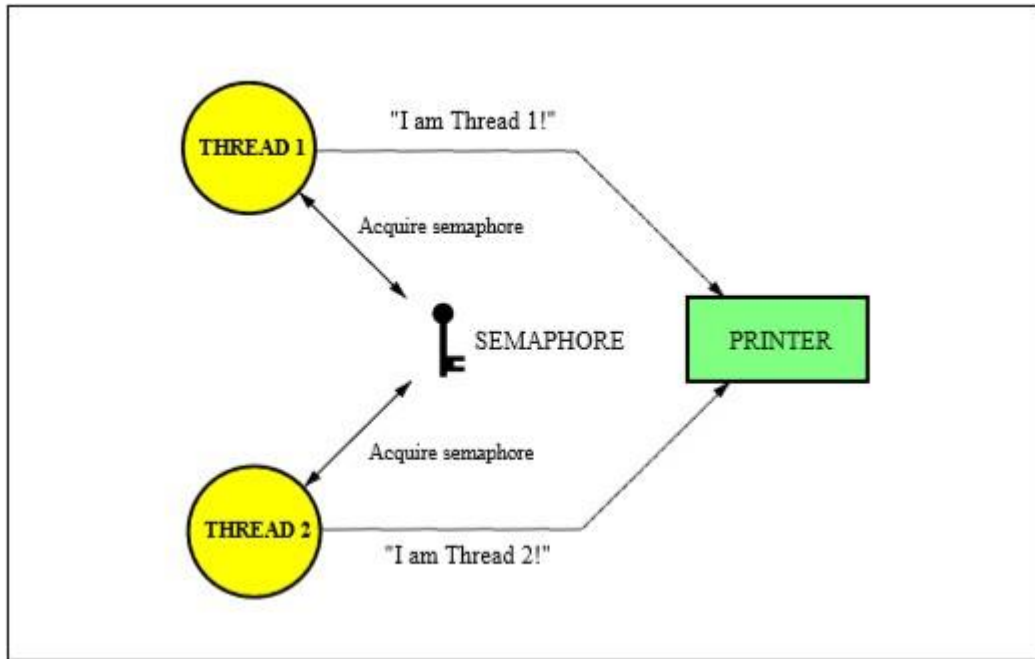


Figure – Using a semaphore to get permission to access a printer

The above example implies that each thread must know about the existence of the semaphore in order to access the resource. There are situations when it is better to encapsulate the semaphore. Each thread would thus not know that it is actually acquiring a semaphore when accessing the resource. For example, the UART port may be used by multiple threads to send commands and receive responses from a PC:

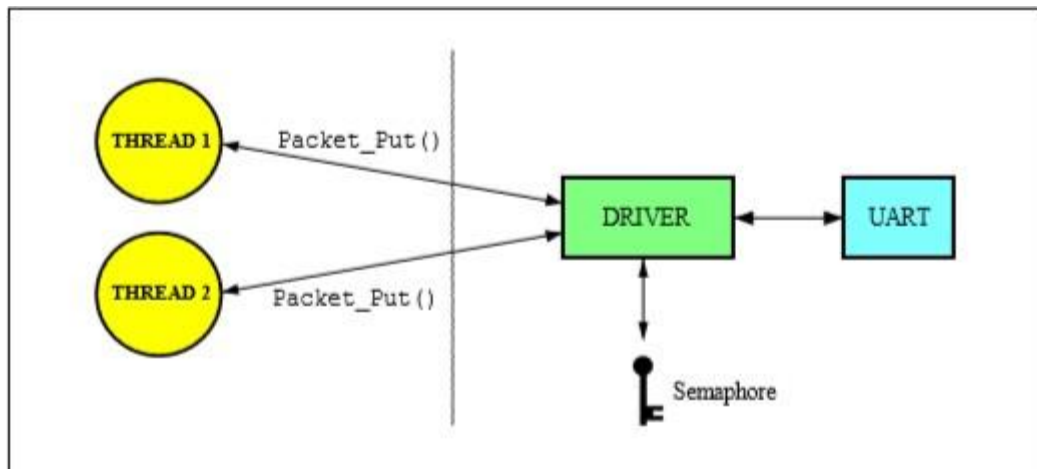


Figure – Hiding a semaphore from threads

The function `Packet_Put()` is called with two arguments: the packet and a timeout in case the device doesn't respond within a certain amount of time

Deadlock (or Deadly Embrace)

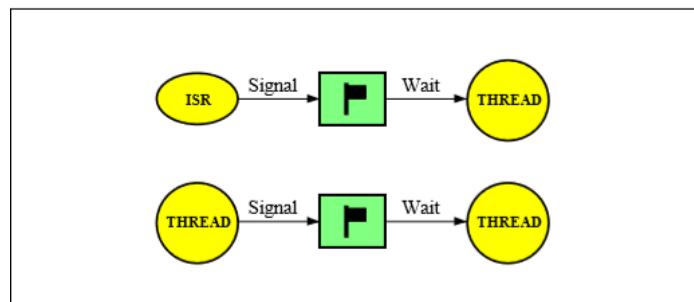
A deadlock, also called a deadly embrace, is a situation in which two threads are each unknowingly waiting for resources held by the other. Assume thread T1 has exclusive access to resource R1 and thread T2 has exclusive access to resource R2. If T1 needs exclusive access to R2 and T2 needs exclusive access to R1, neither thread can continue. They are deadlocked. The simplest way to avoid a deadlock is for threads to:

- acquire all resources before proceeding,
- acquire the resources in the same order, and
- release the resources in the reverse order

Most kernels allow to specify a timeout when acquiring a semaphore. This feature allows a deadlock to be broken. If the semaphore is not available within a certain amount of time, the thread requesting the resource resumes execution. Some form of error code must be returned to the thread to notify it that a timeout occurred. A return error code prevents the thread from thinking it has obtained the resource. Deadlocks generally occur in large multithreading systems, not in embedded systems.

Task Synchronization

A thread can be synchronized with an ISR (or another thread when no data is being exchanged) by using a semaphore as shown in Figure.



Note that, in this case, the semaphore is drawn as a flag to indicate that it is used to signal the occurrence of an event (rather than to ensure mutual exclusion, in which case it would be drawn as a key). When used as a synchronization mechanism, the semaphore is initialized to 0. Using a semaphore for this type of synchronization is called a **unilateral rendezvous**. A thread initiates an I/O operation and waits for the semaphore. When the I/O operation is complete, an ISR (or another thread) signals the semaphore and the thread is resumed.

If the kernel supports counting semaphores, the semaphore would accumulate events that have not yet been processed. Note that more than one thread can be waiting for an event to occur. In this case, the kernel could signal the occurrence of the event either to:

- the highest priority thread waiting for the event to occur or
- the first thread waiting for the event.

Depending on the application, more than one ISR or thread could signal the occurrence of the event. Two threads can synchronize their activities by using two semaphores, as shown in Figure below. This is called a bilateral rendezvous. A bilateral rendezvous is similar to a unilateral rendezvous, except both threads must synchronize with one another before proceeding.

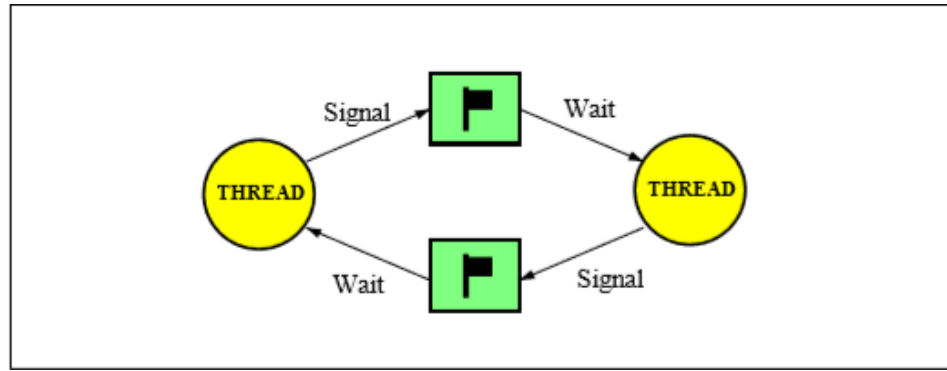


Figure – Threads synchronizing their activities

For example, two threads are executing as shown in Listing below. When the first thread reaches a certain point, it signals the second thread (1) then waits for a return signal (2). Similarly, when the second thread reaches a certain point, it signals the first thread (3) and waits for a return signal (4). At this point, both threads are synchronized with each other. A bilateral rendezvous cannot be performed between a thread and an ISR because an ISR cannot wait on a semaphore:

```

void Thread1(void)
{
    for (;;)
    {
        Perform operation 1;
        Signal thread #2;           (1)
        Wait for signal from thread #2; (2)
        Continue operation 1;
    }
}

void Thread2(void)
{
    for (;;)
    {
        Perform operation 2;
        Signal thread #1;           (3)
        Wait for signal from thread #1; (4)
        Continue operation 2;
    }
}
  
```

Interthread Communication

It is sometimes necessary for a thread or an ISR to communicate information to another thread. This information transfer is called interthread communication. Information may be communicated between threads in two ways: through global data or by sending messages.

When using global variables, each thread or ISR must ensure that it has exclusive access to the variables. If an ISR is involved, the only way to ensure exclusive access to the common variables is to disable interrupts. If two threads are sharing data, each can gain exclusive access to the variables either by disabling and enabling interrupts or with the use of a semaphore (as we have seen). Note that a thread can only communicate information to an ISR by using global variables. A thread is not aware when a global variable is changed by an ISR, unless the ISR signals the thread by using a semaphore or unless the thread polls the contents of the variable periodically.

To correct this situation, we should consider using either a message mailbox or a message queue.

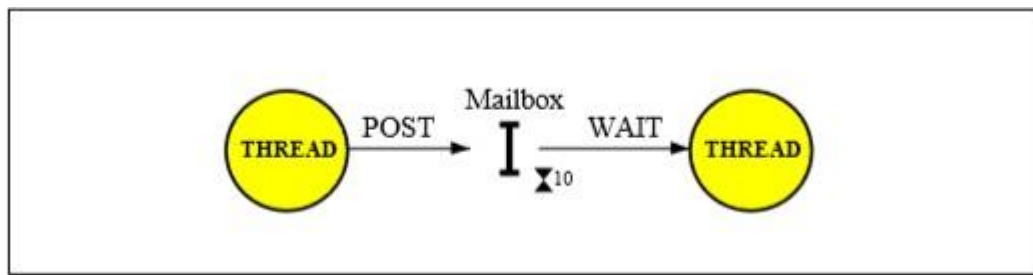


Figure – Message mailbox

Messages can be sent to a thread through kernel services. A Message Mailbox, also called a message exchange, is typically a pointer-size variable. Through a service provided by the kernel, a thread or an ISR can deposit a message (the pointer) into this mailbox. Similarly, one or more threads can receive messages through a service provided by the kernel. Both the sending thread and receiving thread agree on what the pointer is actually pointing to.

A waiting list is associated with each mailbox in case more than one thread wants to receive messages through the mailbox. Kernels typically provide the following mailbox services:

- Initialize the contents of a mailbox. The mailbox initially may or may not contain a message.
- Deposit a message into the mailbox (POST).
- Wait for a message to be deposited into the mailbox (WAIT).
- Get a message from a mailbox if one is present, but do not suspend the caller if the mailbox is empty (ACCEPT). If the mailbox contains a message, the message is extracted from the mailbox. A return code is used to notify the caller about the outcome of the call.

Message mailboxes can also simulate binary semaphores. A message in the mailbox indicates that the resource is available, and an empty mailbox indicates that the resource is already in use by another thread.

Message Queues

A message queue is used to send one or more messages to a thread. A message queue is basically an array of mailboxes. Through a service provided by the kernel, a thread or an ISR can deposit a message (the pointer) into a message queue. Similarly, one or more threads can receive messages through a service provided by the kernel. Both the sending thread and receiving thread agree as to what the pointer is actually pointing to. Generally, the first message inserted in the queue will be the first message extracted from the queue (FIFO).

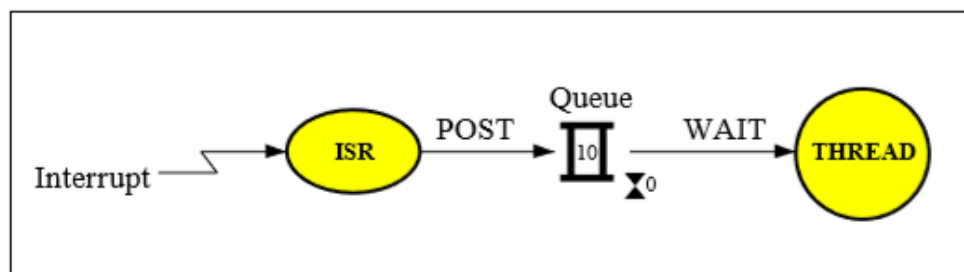


Figure – Message queue

As with the mailbox, a waiting list is associated with each message queue, in case more than one thread is to receive messages through the queue. A thread desiring a message from an empty queue is suspended and placed on the waiting list until a message is received. Typically, the kernel allows the thread waiting for a message to specify a timeout. If a message is not received before the timeout expires, the requesting thread is made ready to run and an error code (indicating a timeout has occurred) is returned to it. When a message is deposited into the queue, either the highest priority thread or the first thread to wait for the message is given the message. Kernels typically provide the message queue services listed below.

- Initialize the queue. The queue is always assumed to be empty after initialization.
- Deposit a message into the queue (POST).
- Wait for a message to be deposited into the queue (WAIT).
- Get a message from a queue if one is present, but do not suspend the caller if the queue is empty (ACCEPT). If the queue contains a message, the message is extracted from the queue. A return code is used to notify the caller about the outcome of the call.

Interrupts

Clock Tick

A clock tick is a special interrupt that occurs periodically. This interrupt can be viewed as the system's heartbeat. The time between interrupts is application specific and is generally between 1 and 200 ms. The clock tick interrupt allows a kernel to delay threads for an integral number of clock ticks and to provide timeouts when threads are waiting for events to occur. The faster the tick rate, the higher the overhead imposed on the system.

All kernels allow threads to be delayed for a certain number of clock ticks. The resolution of delayed threads is one clock tick; however, this does not mean that its accuracy is one clock tick.

Memory Requirement

If we are designing a foreground / background system, the amount of memory required depends solely on application code. With a multithreading kernel, things are quite different. To begin with, a kernel requires extra code space (Flash). The size of the kernel depends on many factors. Depending on the features provided by the kernel, we can expect anywhere from 1 to 100 KiB. A minimal kernel for a 32-bit CPU that provides only scheduling, context switching, semaphore management, delays, and timeouts should require about 1 to 3 KiB of code space.

Because each thread runs independently of the others, it must be provided with its own stack area (RAM). As a designer, you must determine the stack requirement of each thread as closely as possible (this is sometimes a difficult undertaking). The stack size must not only account for the thread requirements (local variables, function calls, etc.), it must also account for maximum interrupt nesting (saved registers, local storage in ISRs, etc.). Depending on the target processor and the kernel used, a separate stack can be used to handle all interrupt-level code. This is a desirable feature because the stack requirement.

For each thread can be substantially reduced. Another desirable feature is the ability to specify the stack size of each thread on an individual basis. Conversely, some kernels require that all thread stacks be the same size. All kernels require extra RAM to maintain internal variables, data structures, queues, etc. The total RAM required if the kernel does not support a separate interrupt stack is given by:

Total RAM requirements = Application code requirements + Data space (i.e., RAM) needed by the kernel + SUM (thread stacks + MAX (ISR nesting))

Unless we have large amounts of RAM to work with, you need to be careful how you use the stack space. To reduce the amount of RAM needed in an application, we must be careful how you use each thread's stack for:

- large arrays and structures declared locally to functions and ISRs,
- function (i.e., subroutine) nesting,
- interrupt nesting,
- library functions stack usage, and
- Function calls with many arguments.

To summarize, a multithreading system requires more code space (Flash) and data space (RAM) than a foreground / background system. The amount of extra Flash depends only on the size of the kernel, and the amount of RAM depends on the number of threads in system.

Typical Semaphore Use

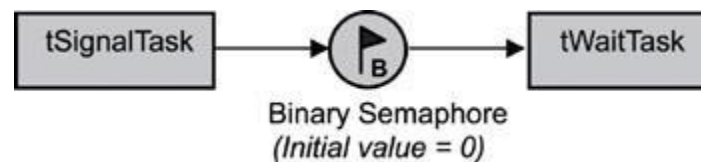
Semaphores are useful either for synchronizing execution of multiple tasks or for coordinating access to a shared resource. The following examples and general discussions illustrate using different types of semaphores to address common synchronization design requirements effectively, as listed:

- wait-and-signal synchronization,
- multiple-task wait-and-signal synchronization,
- credit-tracking synchronization,
- single shared-resource-access synchronization,
- recursive shared-resource-access synchronization, and
- multiple shared-resource-access synchronization.

Note that, for the sake of simplicity, not all uses of semaphores are listed here. Also, later chapters of this book contain more advanced discussions on the different ways that mutex semaphores can handle priority inversion.

Wait-and-Signal Synchronization

Two tasks can communicate for the purpose of synchronization without exchanging data. For example, a binary semaphore can be used between two tasks to coordinate the transfer of execution control, as shown in figure below.



Multiple-Task Wait-and-Signal Synchronization

When coordinating the synchronization of more than two tasks, use the flush operation on the task-waiting list of a binary semaphore, as shown in Figure below.

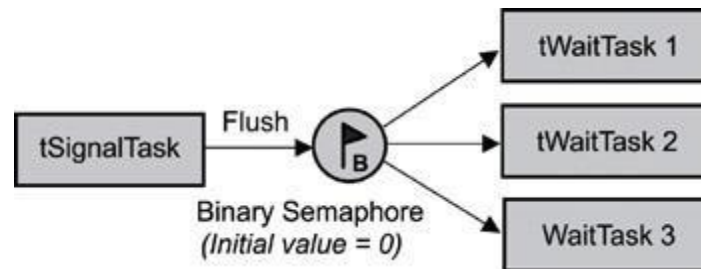


Figure: Wait-and-signal synchronization between multiple tasks.

As in the previous case, the binary semaphore is initially unavailable (value of 0). The higher priority `tWaitTasks` 1, 2, and 3 all do some processing; when they are done, they try to acquire the unavailable semaphore and, as a result, block. This action gives `tSignalTask` a chance to complete its processing and execute a flush command on the semaphore, effectively unblocking the three `tWaitTasks`.

Credit-Tracking Synchronization

Sometimes the rate at which the signaling task executes is higher than that of the signaled task. In this case, a mechanism is needed to count each signaling occurrence. The counting semaphore provides just this facility. With a counting semaphore, the signaling task can continue to execute and increment a count at its own pace, while the wait task, when unblocked, executes at its own pace, as shown in figure below.

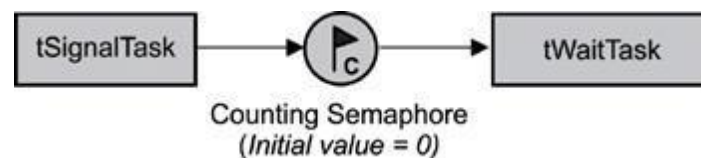


Figure : Credit-tracking synchronization between two tasks.

Again, the counting semaphore's count is initially 0, making it unavailable. The lower priority `tWaitTask` tries to acquire this semaphore but blocks until `tSignalTask` makes the semaphore available by performing a release on it. Even then, `tWaitTask` will wait in the ready state until the higher priority `tSignalTask` eventually relinquishes the CPU by making a blocking call or delaying itself.

Single Shared-Resource-Access Synchronization

One of the more common uses of semaphores is to provide for mutually exclusive access to a shared resource. A shared resource might be a memory location, a data structure, or an I/O device-essentially anything that might have to be shared between two or more concurrent threads of execution. A semaphore can be used to serialize access to a shared resource, as shown in figure below.

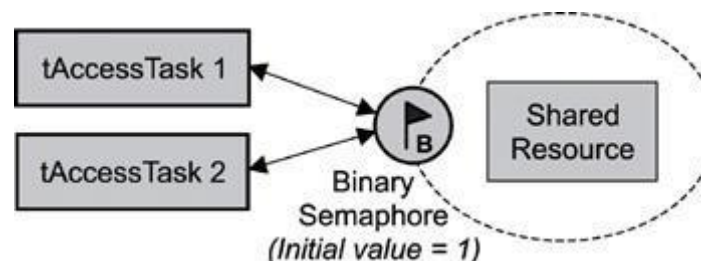


Figure: Single shared-resource-access synchronization.

In this scenario, a binary semaphore is initially created in the available state (value = 1) and is used to protect the shared resource. To access the shared resource, task 1 or 2 needs to first successfully acquire the binary semaphore before reading from or writing to the shared resource.

Recursive Shared-Resource-Access Synchronization

Sometimes a developer might want a task to access a shared resource recursively. This situation might exist if *tAccessTask* calls Routine A that calls Routine B, and all three need access to the same shared resource, as shown in figure below.

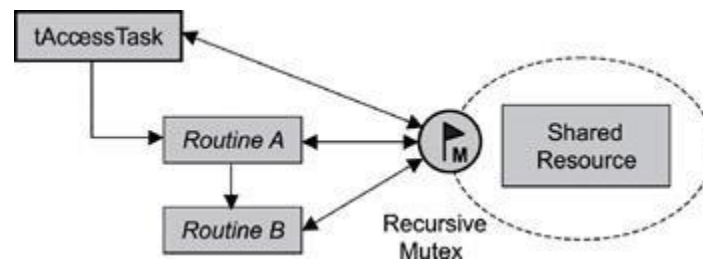


Figure: Recursive shared- resource-access synchronization.

If a semaphore were used in this scenario, the task would end up blocking, causing a deadlock. When a routine is called from a task, the routine effectively becomes a part of the task. When Routine A runs, therefore, it is running as a part of *tAccessTask*. Routine A trying to acquire the semaphore is effectively the same as *tAccessTask* trying to acquire the same semaphore. In this case, *tAccessTask* would end up blocking while waiting for the unavailable semaphore that it already has.

One solution to this situation is to use a recursive mutex. After *tAccessTask* locks the mutex, the task owns it. Additional attempts from the task itself or from routines that it calls to lock the mutex succeed. As a result, when Routines A and B attempt to lock the mutex, they succeed without blocking.

Multiple Shared-Resource-Access Synchronization

For cases in which multiple equivalent shared resources are used, a counting semaphore comes in handy, as shown in Figure

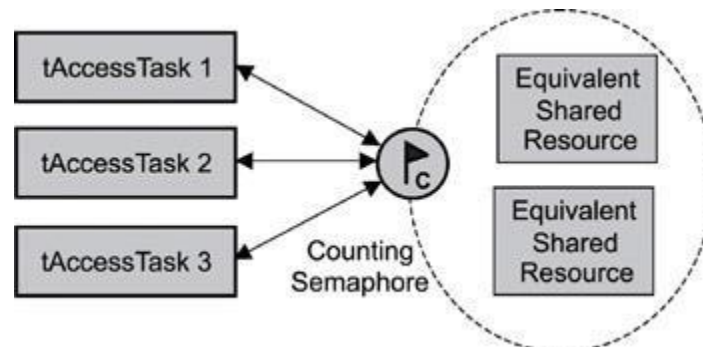


Figure: Single shared-resource-access synchronization.

Note that this scenario does not work if the shared resources are not equivalent. The counting semaphore's count is initially set to the number of equivalent shared resources: in this example, 2. As a result, the first two tasks requesting a semaphore token are successful. However, the third task ends up blocking until one of the previous two tasks releases a semaphore token.

Memory Management

Embedded systems developers commonly implement custom memory-management facilities on top of what the underlying RTOS provides. Understanding memory management is therefore an important aspect of developing for embedded systems.

Knowing the capability of the memory management system can aid application design and help avoid pitfalls. For example, in many existing embedded applications, the dynamic memory allocation routine, `malloc`, is called often. It can create an undesirable side effect called memory fragmentation. This generic memory allocation routine, depending on its implementation, might impact an application's performance. In addition, it might not support the allocation behavior required by the application.

Many embedded devices (such as PDAs, cell phones, and digital cameras) have a limited number of applications (tasks) that can run in parallel at any given time, but these devices have small amounts of physical memory onboard. Larger embedded devices (such as network routers and web servers) have more physical memory installed, but these embedded systems also tend to operate in a more dynamic environment, therefore making more demands on memory. Regardless of the type of embedded system, the common requirements placed on a memory management system are minimal fragmentation, minimal management overhead, and deterministic allocation time.

Dynamic Memory Allocation in Embedded Systems

It is known that the program code, program data, and system stack occupy the physical memory after program initialization completes. Either the RTOS or the kernel typically uses the remaining physical memory for dynamic memory allocation. This memory area is called the *heap*. Memory management in the context of this chapter refers to the management of a contiguous block of physical memory, although the concepts introduced in this apply to the management of non-contiguous memory blocks as well. These concepts also apply to the management of various types of physical memory. In general, a memory management facility maintains internal information for a heap in a reserved memory area called the *control block*. Typical internal information includes:

- the starting address of the physical memory block used for dynamic memory allocation,
- the overall size of this physical memory block, and
- the allocation table that indicates which memory areas are in use, which memory areas are free, and the size of each free region.

Memory Fragmentation and Compaction

In the example implementation, the heap is broken into small, fixed-size blocks. Each block has a unit size that is power of two to ease translating a requested size into the corresponding required number of units. In this example, the unit size is 32 bytes. The dynamic memory allocation function, `malloc`, has an input parameter that specifies the size of the allocation request in bytes. `malloc` allocates a larger block, which is made up of one or more of the smaller, fixed-size blocks. The size of this larger memory block is at least as large as the requested size; it is the closest to the multiple of the unit size. For example, if the allocation requests 100 bytes, the returned block has a size of 128 bytes (4 units x 32 bytes/unit). As a result, the requestor does not use 28 bytes of the allocated memory, which is called memory fragmentation. This specific form of fragmentation is called internal fragmentation because it is internal to the allocated block.

The allocation table can be represented as a bitmap, in which each bit represents a 32-byte unit. Figure shows the states of the allocation table after a series of invocations of the `malloc` and `free` functions. In this example, the heap is 256 bytes.

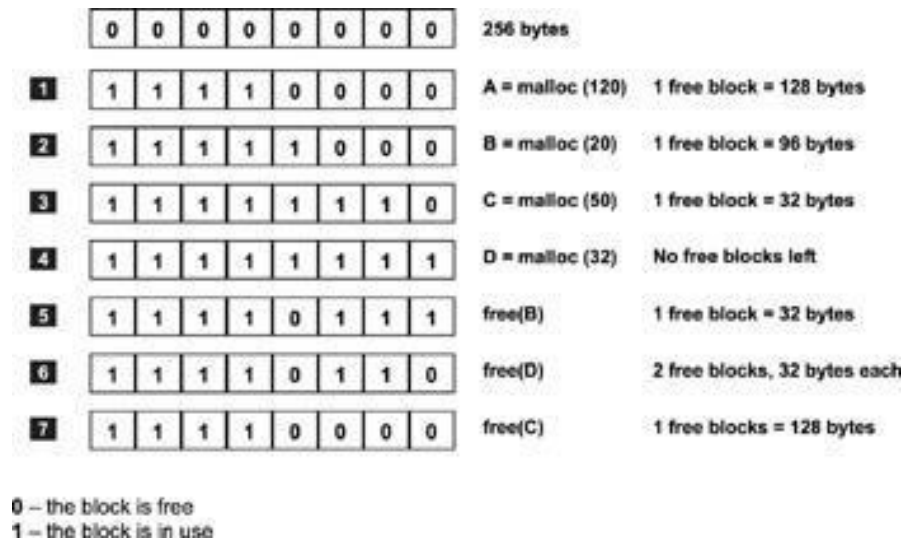


Figure: States of a memory allocation map.

Step 6 shows two free blocks of 32 bytes each. Step 7, instead of maintaining three separate free blocks, shows that all three blocks are combined to form a 128-byte block. Because these blocks have been combined, a future allocation request for 96 bytes should succeed.

Figure below shows another example of the state of an allocation table. Note that two free 32-byte blocks are shown. One block is at address 0x10080, and the other at address 0x101C0, which cannot be used for any memory allocation requests larger than 32 bytes. Because these isolated blocks do not contribute to the contiguous free space needed for a large allocation request, their existence makes it more likely that a large request will fail or take too long. The existence of these two trapped blocks is considered external fragmentation because the fragmentation exists in the table, not within the blocks themselves. One way to eliminate this type of fragmentation is to compact the area adjacent to these two blocks. The range of memory content from address 0x100A0 (immediately following the first free block) to address 0x101BF (immediately preceding the second free block) is shifted 32 bytes lower in memory, to the new range of 0x10080 to 0x1019F, which effectively combines the two free blocks into one 64-byte block. This new free block is still considered memory fragmentation if future allocations are potentially larger than 64 bytes. Therefore, memory compaction continues until all of the free blocks are combined into one large chunk.

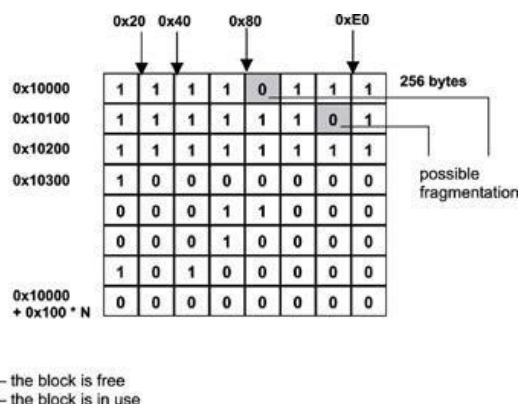


Figure : Memory allocation map with possible fragmentation.

Several problems occur with memory compaction. It is time-consuming to transfer memory content from one location to another. The cost of the copy operation depends on the length of the contiguous blocks in use. The tasks that currently hold ownership of those memory blocks are prevented from accessing the contents of those memory locations until the transfer operation completes. Memory compaction is almost never done in practice in embedded designs. The free memory blocks are combined only if they are immediate neighbors, as illustrated in Figure above.

Memory compaction is allowed if the tasks that own those memory blocks reference the blocks using virtual addresses. Memory compaction is not permitted if tasks hold physical addresses to the allocated memory blocks.

In many cases, memory management systems should also be concerned with architecture-specific memory alignment requirements. *Memory alignment* refers to architecture-specific constraints imposed on the address of a data item in memory. Many embedded processor architectures cannot access multi-byte data items at any address. For example, some architecture requires multi-byte data items, such as integers and long integers, to be allocated at addresses that are a power of two. Unaligned memory addresses result in bus errors and are the source of memory access exceptions.

Some conclusions can be drawn from this example. An efficient memory manager needs to perform the following chores quickly:

- Determine if a free block that is large enough exists to satisfy the allocation request. This work is part of the malloc operation.
- Update the internal management information. This work is part of both the malloc and free operations.
- Determine if the just-freed block can be combined with its neighboring free blocks to form a larger piece. This work is part of the free operation.

The structure of the allocation table is the key to efficient memory management because the structure determines how the operations listed earlier must be implemented. The allocation table is part of the overhead because it occupies memory space that is excluded from application use. Consequently, one other requirement is to minimize the management overhead.

Chapter4

VHDL stands for very high-speed integrated circuit hardware description language. It is a programming language used to model a digital system by dataflow, behavioral and structural style of modeling. This language was first introduced in 1981 for the department of Defense (DoD) under the VHSIC program.

- The VHSIC Hardware Description Language (VHDL) is an industry standard language used to describe hardware from the abstract to concrete level.
- The language not only defines the syntax but also defines very clear simulation semantics for each language construct.
- It is strong typed language and is often verbose to write.
- Provides extensive range of modeling capabilities, it is possible to quickly assimilate a core subset of the language that is both easy and simple to understand without learning the more complex features.

Why Use VHDL?

- Quick Time-to-Market
 - Allows designers to quickly develop designs requiring tens of thousands of logic gates.
 - Provides powerful high-level constructs for describing complex logic.
 - Supports modular design methodology and multiple levels of hierarchy.
- One language for design and simulation.
- Allows creation of device-independent designs that are portable to multiple vendors. Good for ASIC Migration.
- Allows user to pick any synthesis tool, vendor, or device.

Basic Features of VHDL:

- Concurrency.
 - Supports Sequential Statements.
 - Supports for Test & Simulation.
-

- Strongly Typed Language.
- Supports Hierarchies.
- Supports for Vendor Defined Libraries.
- Supports Multivalued Logic.

Concurrency

- VHDL is a concurrent language.
- HDL differs with Software languages with respect to Concurrency only.
- VHDL executes statements at the same time in parallel, as in Hardware.

Supports Sequential Statements

- VHDL supports sequential statements also, it executes one statement at a time in sequence only.
- As the case with any conventional languages.
- Example:

```
if a='1' then  
  y<='0';  
else y<='1';  
end if ;
```

Supports for Test & Simulation

- To ensure that design is correct as per the specifications, the designer has to write another program known as "TEST BENCH".
- It generates a set of test vectors and sends them to the design under test (DUT).
- Also gives the responses made by the DUT against a specifications for correct results to ensure the functionality.

Strongly Typed Language

- VHDL allows LHS & RHS operators of same type.
 - Different types in LHS & RHS is illegal in VHDL.
-

- Allows different type assignment by conversion.
- Example:

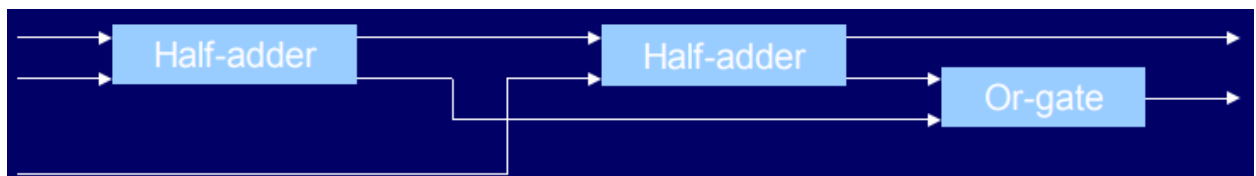
```

A : in std_logic_vector(3 downto 0).
B : out std_logic_vector(3 downto 0).
C : in bit_vector (3 downto 0).
B <= A; --perfect.
B <= C; --type miss match, syntax error.

```

Supports Hierarchies:

- Hierarchy can be represented using VHDL.
- Consider example of a Full-adder which is the top-level module, being composed of three lower level modules i.e. half-Adder and OR gate.
- Example :



Levels of Abstraction:

Data Flow level

- In this style of modeling the flow of data through the entity is expressed using concurrent signal assignment statements.

Structural level

- In this style of modeling the entity is described as a set of interconnected statements.

Behavioral level.

- This style of modeling specifies the behavior of an entity as a set of statements that are executed sequentially in the specified order.

VHDL Identifiers:

- Identifiers are used to name items in a VHDL model.
- A basic identifier may contain only capital 'A' - 'Z', 'a' - 'z', '0' - '9', underscore character '_'.
- Must start with a alphabet.
- May not end with a underscore character.
- Must not include two successive underscore characters.
- Reserved word cannot be used as identifiers.
- VHDL is not case sensitive.

Objects:

There are three basic object types in VHDL.

- Signal: represents interconnections that connect components and ports.
- Variable: used for local storage within a process.
- Constant: a fixed value.

The object type could be a scalar or an array.

Data Types in VHDL:

Type

- Is a name which is associated with a set of values and a set of operations.

Major Types

- Major types
- Composite Types

Scalar Types

Integer

- Maximum range of integer is tool dependent type integer is range implementation_defined.
- For example:

constant loop_no : integer := 345;

Signal my_int : integer range 0 to 255;

Floating Point:

- Can be either positive or negative.
- Exponents have to be integer.

type real is range implementation_defined

Physical

- Predefined type “Time” used to specify delays.
- Example : type TIME is range -2147483647 to 2147483647

Enumeration

- Values are defined in ascending order.
- Example:
- type alu is (pass, add, subtract, multiply, divide)

Composite Types

There are two composite types

Array:

- Contain many elements of the same type.
- Array can be either single or multidimensional.
- Single dimensional array are synthesizable.
- The synthesis of multidimensional array depends upon the synthesizer being used.

Record:

- Contain elements of different types.

The Std Logic Type:

- It is a data type defined in the std_logic_1164 package of IEEE library.
- It is an enumerated type and is defined as

type std_logic is ('U', 'X', '0', '1', 'Z', 'W', 'L', 'H', '-')

'u'	unspecified
'x'	unknown
'0'	strong zero
'1'	strong one
'z'	high impedance
'w'	weak unknown
'l'	weak zero
'h'	weak one
'-'	don't care

Alias:

- Alias is an alternative name assigned to part of an object simplifying its access.
- Syntax :

alias alias_name : subtype is name;

- Examples:

signal inst : std_logic_vector(7 downto 0);

alias opcode : std_logic_vector(3 downto 0) is inst (7 downto 4);

alias srce : std_logic_vector(1 downto 0) is inst(3 downto 2);

alias dest : std_logic_vector(1 downto 0) is inst (1 downto 0);

Signal Array:

- A set of signals may also be declared as a signal array which is a concatenated set of signals.
- This is done by defining the signal of type bit_vector or std_logic_vector.
- bit_vector and std_logic_vector are types defined in the ieee.std_logic_1164 package.

- Signal array is declared as : < Type > < range >.
- Example:

```
signal data1:bit_vector(1 downto 0);
signal data2: std_logic_vector(7 down to 0);
signal address : std_logic_vector(0 to 15);
```

Subtype

- It is a type with a constraint
- Useful for range checking and for imposing additional constraints on types.

Syntax:

```
subtype subtype_name is base_type range range_constraint;
```

- For example: subtype DIGITS is integer range 0 to 9;

Operators

Predefined VHDL operators can be grouped into seven classes:

1. binary logical operators: **and or nand nor xor xnor**

- **and** logical and result is boolean,
- **nand** logical complement of and result is boolean,
- **nor** logical complement of or result is boolean,
- **xor** logical exclusive or result is boolean,
- **xnor** logical complement of exclusive or result is boolean,

2. relational operators:

- = test for equality, result is boolean
- /= test for inequality, result is Boolean
- < test for less than, result is boolean
- <= test for less than or equal, result is boolean
- > test for greater than, result is Boolean

- `>=` test for greater than or equal, result is Boolean
3. shift operators:
- `sll` shift left logical,
 - `srl` shift right logical,
 - `sla` shift left arithmetic,
 - `sra` shift right arithmetic,
 - `rol` rotate left,
 - `ror` rotate right,
4. adding operators:
- `+` addition, numeric + numeric, result numeric
 - `-` subtraction, numeric - numeric, result numeric
 - `&` concatenation, array or element & array or element, result array
5. unary sign operators:
- `+` unary plus, + numeric, result numeric
 - `-` unary minus, - numeric, result numeric
6. multiplying operators:
- `*` multiplication, numeric * numeric, result numeric
 - `/` division, numeric / numeric, result numeric
 - `mod` modulo, integer mod integer, result integer
 - `rem` remainder, integer rem integer, result integer
7. miscellaneous operators:
- `abs` absolute value, abs numeric, result numeric
 - `not` complement, not logic or boolean, result same
 - `**` exponentiation, numeric ** integer, result numeric

Multi-Dimensional Arrays

Syntax

type array_name is array (index_range , index_range) of element_type;

For example:

type memory is array (3 downto 0, 7 downto 0);

For synthesizers which do not accept multidimensional arrays, one can declare two unidimensional arrays.

For example:

type byte is array (7 downto 0) of std_logic;

type mem is array (3 downto 0) of byte;

Data Flow Modeling:

Dataflow Level

8. A Dataflow model specifies the functionality of the entity without explicitly specifying its structure.
9. This functionality shows the flow of information through the entity, which is expressed primarily using concurrent signal assignment statements and block statements.
10. The primary mechanism for modeling the dataflow behavior of an entity is using the concurrent signal assignment statement.

Entity

11. Entity describes the design interface.
12. The interconnections of the design unit with the external world are enumerated.
13. The properties of these interconnections are defined.

Entity declaration:

```
Entity <entity_name> is
    Port (port_name: <mode> <type>
    .....
);
```



End <entity_name>;

14. There are four modes for the ports in VHDL

in, out, inout, buffer

15. These modes describe the different kinds of interconnections that the port can have with the external circuitry.

16. Sample program:

Entity andgate is

Port (c : out bit;

a: in bit;

b : in bit

);

End andgate;

Architecture:

17. Architecture defines the functionality of the entity.

18. It forms the body of the VHDL code.

19. An architecture belongs to a specific entity.

20. Various constructs are used in the description of the architecture.

21. architecture declaration:

```
architecture <architecture_name> of
  <entity _ name> is
    <declarations>
  Begin
    <vhdl statements>
  end <architecture_name>;
```

22. Example of a VHDL architecture is

```
architecture arc_andgate of andgate is
begin
    c <= a and b;
end arc_andgate;
```

Write the VHDL code for AND gate.

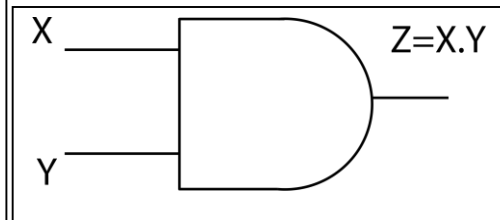
```
Library ieee;
use ieee.std_logic_1164.all;

Entity andgate is
Port(
    x, y: in std_logic;
    z : out std_logic
);

End andgate;

architecture arc_andgate of andgate is
begin
    z <= x and y;
end arc_andgate;
```

X	Y	Z
0	0	0
0	1	0
1	0	0
1	1	1



Write the VHDL code for full adder circuit

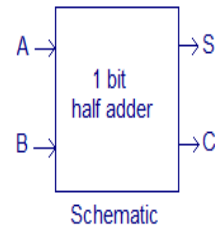
```
Library ieee;
use ieee.std_logic_1164.all;

Entity half_adder is
Port(
    a, b: in std_logic;
    c, s : out std_logic;
);
End half_adder;

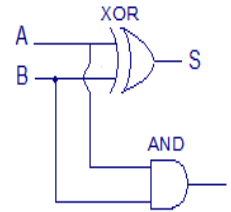
architecture arc_half_adder of half_adder is
begin
    c<= x and y;
    s<= x xor y;
end arc_half_adder;
```

Inputs		Outputs	
A	B	S	C
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1

Truth table



Schematic



Realization

Signals

23. Syntax: signal signal_name <list of signals> : type := initial_value;
24. Equivalent to wires.
25. Connect design entities together and communicate changes in values within a design.
26. Computed value is assigned to signal after a specified delay called as Delta Delay.
27. Signals can be declared in an entity (it can be seen by all the architectures), in an architecture (local to the architecture), in a package (globally available to the user of the package) or as a parameter of a subprogram (I.e. function or procedure).
28. Signals have three properties attached to it.
Type and Type attributes, value, Time (It has a history).
29. Signal assignment be done by using assignment operator: '<='.
30. Signal assignment is concurrent outside a process & sequential within a process.

```

Library ieee;

use ieee.std_logic_1164.all;

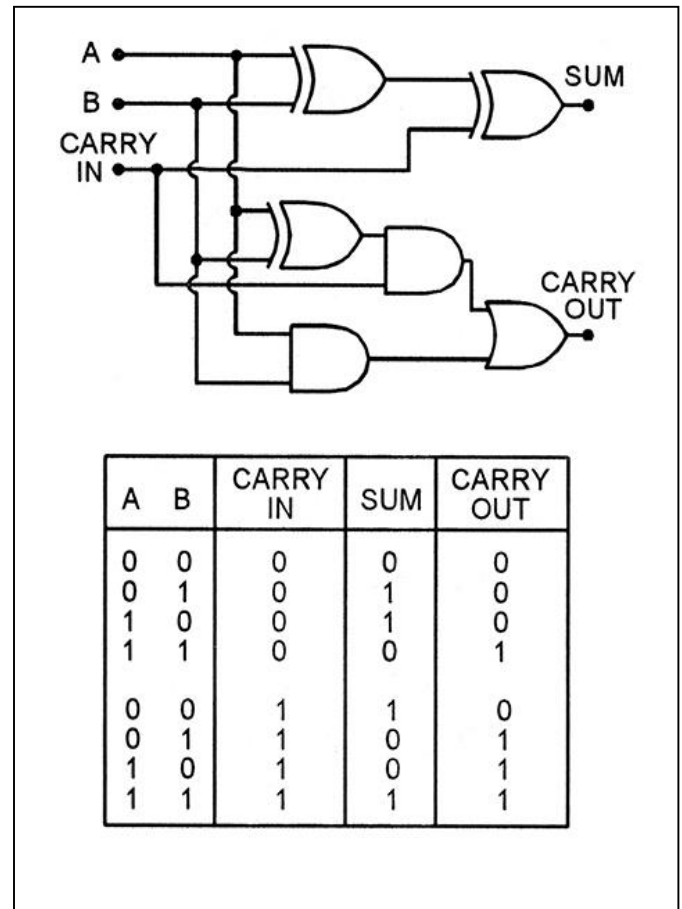
Entity full_adder is
Port(
    a, b, c: in std_logic;
    carry, sum : out std_logic;
);
End full_adder;

architecture arc_full_adder of full_adder is
signal x, y, z : std_logic;
begin
x<= a xor b;
sum<= x xor c;

y<= x and c;
z<= a and b;
carry <= y or z;
end arc_full_adder;

```

Write the VHDL code for full adder circuit.



Structural Modeling:

31. An entity is modeled as a set of components connected by signals, that is, as a netlist.
32. The behavior of the entity is not explicitly apparent from its model.
33. The component instantiation statement is the primary mechanism used for describing such a model of an entity.
34. A component instantiated in a structural description must first be declared using a component declaration.
35. A larger design entity can call a smaller design unit in it.
36. This forms a hierarchical structure.
37. This is allowed by a feature of VHDL called component instantiation.
38. A component is a design entity in itself which is instantiated in the larger entity.
39. A component is a design entity in itself which is instantiated in the larger entity.

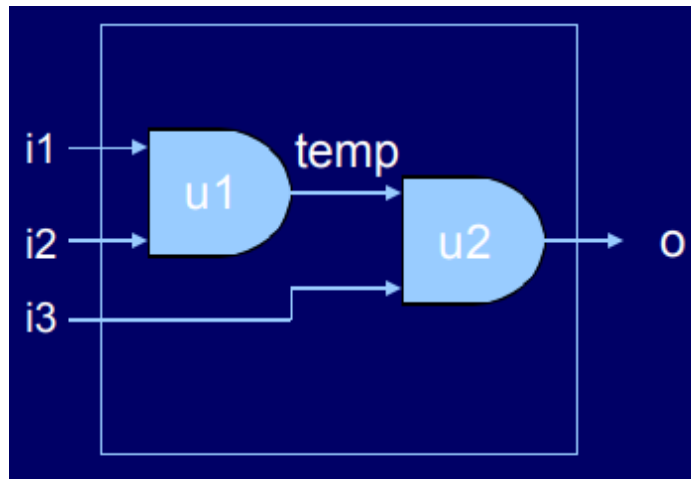
40. Syntax:

```
component <component_name >
port (
    <port_name>: <mode> <type>;
    .....
);
end component;
```

41. The instance (calling) of a component in the entity is described as:

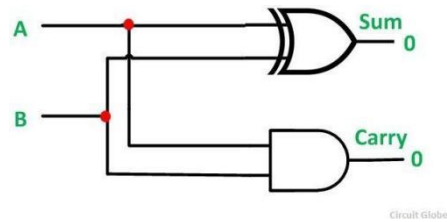
```
<instant_name>: <component_name>port map
    (<association_list>);
```

42. For example: Fig is given as below:



```
Library ieee;
use ieee.std_logic_1164.all;
entity and3gate is
port
(
    o : out std_logic;
    i1 : in std_logic;
    i2 : in std_logic;
    i3 : in std_logic
);
end and3gate;
architecture arc_and3gate of and3gate is
    component andgate is
    port
    (
        c : out std_logic;
        a : in std_logic;
        b : in std_logic
    );
    end component;
    signal temp1 : std_logic;
    begin
        u1:  andgate
            port map(temp1, i1, i2);
        u2:  andgate
            port map(o, temp1, i3);
    end arc_and3gate;
```

#Write a VHDL code to implement the half adder using structural modeling or component.



```
Library ieee;
use ieee.std_logic_1164.all;

entity andgate is
port
(
    c : out std_logic;
    a : in std_logic;
    b: in std_logic;
);
end andgate;

architecture arch_andgate of andgate is
begin
    c<=a and b;
end arch_andgate;

entity xorgate is
port
(
    c : out std_logic;
    a : in std_logic;
    b: in std_logic;
);
end xorgate;

architecture arch_xorgate of xorgate is
begin
```

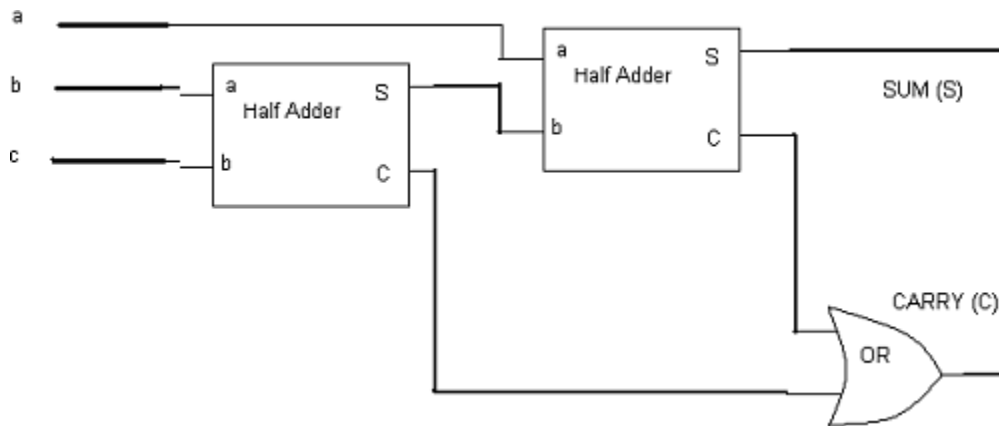
```

c<=a xor b;
end arch_xorgate;
entity halfadder is
port
(
    carry : out std_logic;
    Sum: out std_logic;
    a : in std_logic;
    b: in std_logic;
);
end halfadder;
architecture arch_halfadder of halfadder is
component andgate
port
(
    a, b : in std_logic;
    c: out std_logic;
);
End component;
Component xorgate
(
    a, b : in std_logic;
    c : out std_logic;
);
End component;
Begin
    U0: andgate portmap(carry, a, b);
    U1: xorgate portmap (sum, a, b);
End arch_halfadder;

```

Write the VHDL code to design the full adder circuit using gates as the component.

#Write a VHDL code to implement the full adder using two half adder.



```
Library ieee;
use ieee.std_logic_1164.all;
entity halfadder is
port
(
    s, c : out std_logic;
    x : in std_logic;
    y: in std_logic;
);
end halfadder;
architecture arch_halfadder of halfadder is
begin
s<=x xor y;
c<= x and y;
end arch_halfadder;
entity fulladder is
port
(
    a,b,c : in std_logic;
    Sum, carry : out std_logic;
```

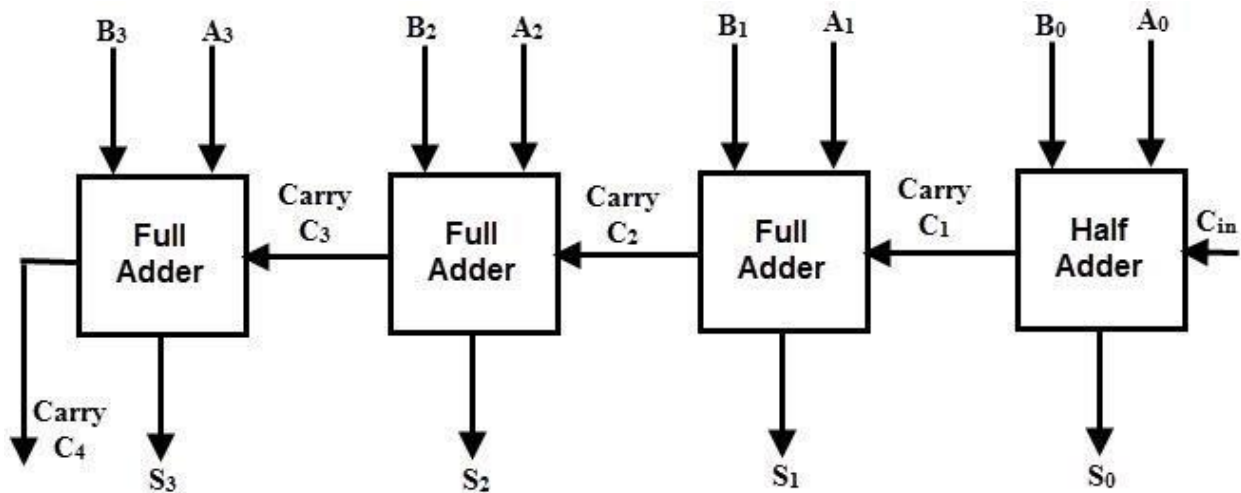
```

);
end fulladder;

architecture arch_fulladder of fulladder is
component halfadder
port
(
    x,y : in std_logic;
    s,c : out std_logic
);
End component;
Signal b1, b2, b3 : std_logic;
begin
U1: halfadder portmap(b,c,b2,b1);
U2: halfadder portmap (b2,a,sum,b3)
carry<= b1 or b2;
end arch_fulladder;

```

Write the VHDL code for 4-bit binary parallel adder.



```

Library ieee;
use ieee.std_logic_1164.all;
entity fulladder is

```

```

port
(
    sum, carry : out std_logic;
    a,b c. : in std_logic;
);
end halfadder;
architecture arc_full_adder of full_adder is
signal x, y, z : std_logic;
begin
x<= a xor b;
sum<= x xor c;
y<= x and c;
z<= a and b;
carry<= y or z;
end arc_full_adder;
entity BPA4 is
port
(
    A,B: in std_logic_vector(3 down to 0);
    Sum_bpa: out std_logic_vector(3 down to 0);
    Cin : in std_logic;
    Cout: out std_logic;
);
end BPA4;
architecture arc_BPA4 of BPA4 is
component fulladder
port
(
    sum, carry : out std_logic;
    a,b c. : in std_logic;
);

```

```

End component;

signal c1, c2, c3 : std_logic;

begin

U0: fulladder portmap(A(0),B(0), Cin, sum(0),c1);

U1: fulladder portmap(A(1),B(1), C1, sum(1),c2);

U2: fulladder portmap(A(2),B(2), C2, sum(2),c2);

U0: fulladder portmap(A(3),B(3), C3, sum(3),Cout);

```

end arc_BPA4;

Concatenation

43. This is the process of combining two signals into a single set which can be individually addressed.
44. The concatenation operator is '&'.
45. A concatenated signal's value is written in double quotes whereas the value of a single bit signal is written in single quotes.

Decision-making statements:

If statements:

```

If (expression) then
    S1
Elseif (expression) then
    S2
Elseif (expression) then
    S3
Elseif (expression) then
    S4
.....
.....
.....
Elseif (expression) then
    Sn
Else
    Sn+1

```

End if;

With-Select

46. The with-select statement is used for selective signal assignment.

47. It is a concurrent statement.

48. Syntax:

```
with expression select:
    target <= expression 1 when choice1
        expression 2 when choice2 . . .
        .....
    .....
    expression N when others;
```

49. Example:

```
entity mux2 is
port
(
    i0, i1 : in bit_vector(1 downto 0);
    y : out bit_vector(1 downto 0);
    sel : in bit
);
end mux2;
architecture behaviour of mux2 is
begin
    with sel select y <= i0 when '0',
        i1 when '1';
end behaviour;
```

When-Else

50. syntax :

```
Signal_name<= expression1 when condition1
else expression2 when condition2
else expression3;
```

51. Example:

```
entity tri_state is
port
```

```

        (      a, enable : in std_logic;
              b : out std_logic
        ); end tri_state;

architecture beh of tri_state is

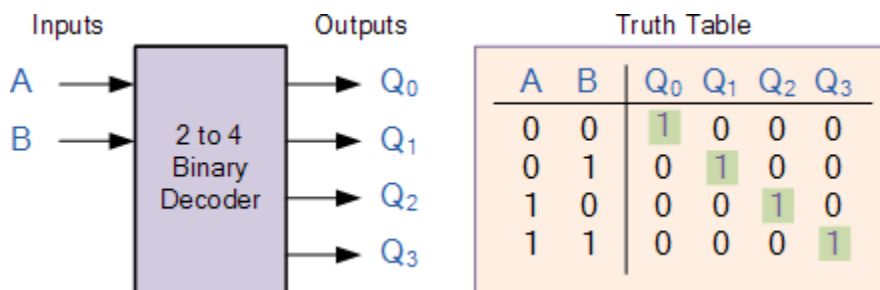
begin

        b <= a when enable = '1'
        else 'Z';

end beh;

```

Write the VHDL code to implement the 2 to 4 decoder.



```

Library ieee;

use ieee.std_logic_1164.all;

entity decoder is

port

(      SW : in std_logic_vector (1 down to 0);
      Q : out std_logic_vector(3 down to 0);

);

end decoder;

architecture arc_decoder of decoder is

begin

        if (SW = "00") then
                Q<= "0001" ;
        elseif (SW = "01") then

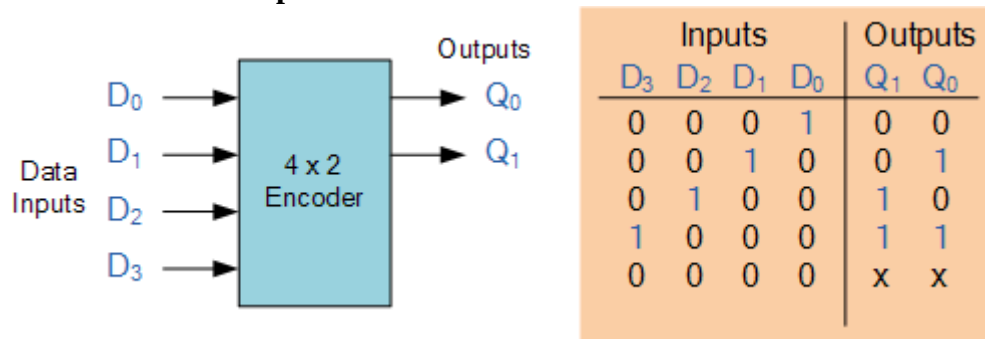
```

```

        Q<= "0010" ;
    elseif (SW = "10") then
        Q<= "0100";
    else
        Q<= "1000";
    Endif;
end arc_decoder;

```

#Write the VHDL code to implement the 4 to 2 encoder.



```

Library ieee;
use ieee.std_logic_1164.all;

entity enecoder is
port
(
    Q : out std_logic_vector (1 down to 0);
    D: in std_logic_vector(3 down to 0);
);
end encoder;

architecture arc_encoder of encoder is
begin
    if (D = "0001") then
        Q<= "00" ;
    elseif (Q = "0001") then
        Q<= "01" ;

```

```

elseif (D= "10") then
    Q<= "10";
else
    Q<= "11";
Endif;
end arc_encoder;

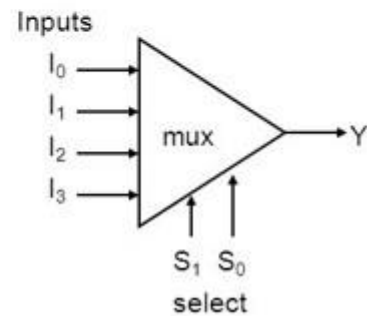
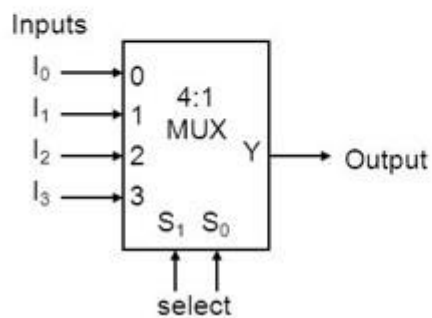
```

Write the VHDL code for implementation of 4 to 1 MUX.

Truth table for a 4-to-1 multiplexer:

I ₀	I ₁	I ₂	I ₃	S ₁	S ₀	Y
d ₀	d ₁	d ₂	d ₃	0	0	d ₀
d ₀	d ₁	d ₂	d ₃	0	1	d ₁
d ₀	d ₁	d ₂	d ₃	1	0	d ₂
d ₀	d ₁	d ₂	d ₃	1	1	d ₃

S ₁	S ₀	Y
0	0	I ₀
0	1	I ₁
1	0	I ₂
1	1	I ₃



```

Library ieee;
use ieee.std_logic_1164.all;
entity MUX is
port
(
    Q : out std_logic;
    I0,I1,I2,I3 : in std_logic;
    SL: in std_logic_vector (1 down to 0);
);
end MUX;

```

architecture arc_MUX of MUX is

begin

if (SL= "00") then

Q<= I0;

elseif (SL= "01") then

Q<= I1 ;

elseif (SL= "10") then

Q<= I2;

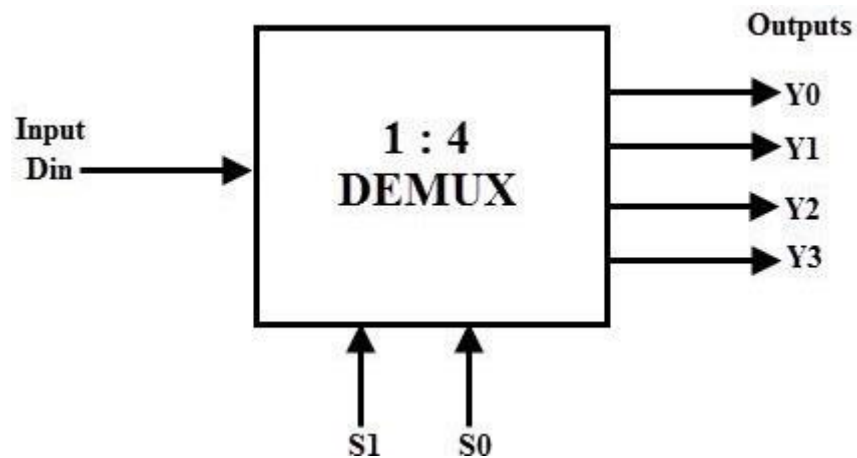
else

Q<= I4;

Endif;

end arc_MUX;

Write the VHDL code for implementation of 4 to 1 MUX.



Library ieee;

use ieee.std_logic_1164.all;

entity DMUX is

port

(Din : in std_logic;

Y0,Y1,Y2,Y3 : out std_logic;

```

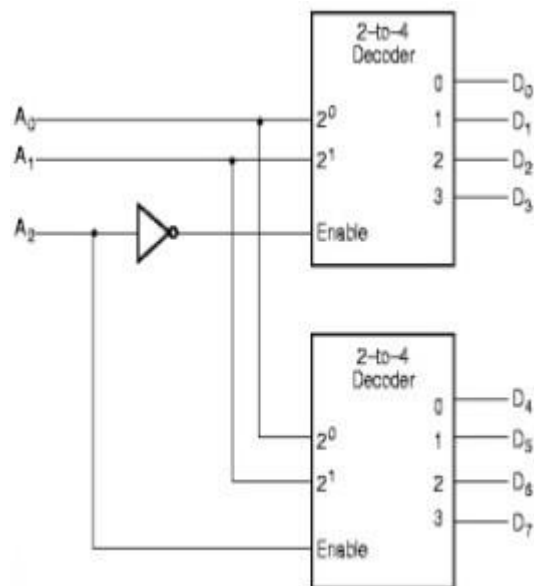
        SL: in std_logic_vector (1 down to 0);

    );
end DMUX;

architecture arc_DMUX of DMUX is
begin
    if (SL= "00") then
        Q0<= Din;
    elseif (SL= "01") then
        Q1<= Din ;
    elseif (SL= "10") then
        Q2<= Din;
    else
        Q3<= Din;
    Endif;
end arc_MUX;

```

Write the VHDL code to implement 3 to 8 decoder using two 2 to 4 decoder.



Do your self

Process Statement:

- A process statement defines an independent sequential process representing the behavior of some portion of the design.
- Simplified Syntax

[process_label:] process [(sensitivity_list)] [is]

process_declarations

begin

sequential_statements

end process [process_label] ;

- The process statement represents the behavior of some portion of the design. It consists of the sequential statements whose execution is made in order defined by the user.
 - Each process can be assigned an optional label.
 - The process declarative part defines local items for the process and may contain declarations of: subprograms, types, subtypes, constants, variables, files, aliases, attributes, use clauses and group declarations. It is not allowed to declare signals or shared variables inside processes.
 - The statements, which describe the behavior in a process, are executed sequentially, in the order in which the designer specifies them. The execution of statements, however, does not terminate with the last statement in the process, but is repeated in an infinite loop. The loop can be *suspended* and *resumed* with *wait* statements. When the next statement to be executed is a *wait* statement, the process suspends its execution until a condition supporting the wait statement is met. See respective topics for details.
 - A process declaration may contain optional sensitivity list. The list contains identifiers of signals to which the process is sensitive. A change of a value of any of those signals causes the suspended process to resume. A sensitivity list is a full equivalent of a *wait on sensitivity_list* statement at the end of the process. It is not allowed, however, to use wait statements and sensitivity list in the same process. In addition, if a process with a sensitivity list calls a procedure, then the procedure cannot contain any wait statements.
-

Sequential Circuits-Gated D Latch

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY flipflop IS
    PORT ( D, Clock : IN    STD_LOGIC ;
          Q          : OUT  STD_LOGIC);
END flipflop ;
```

ARCHITECTURE behavioral OF flipflop IS

BEGIN

PROCESS (**Clock**)

BEGIN

IF **Clock'EVENT AND Clock = '1'** THEN

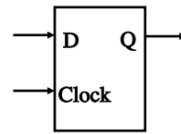
Q <= D ;

END IF ;

END PROCESS ;

END behavioral ;

Graphical symbol

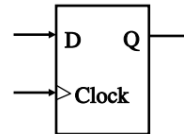


Truth table

Clock	D	Q(t+1)
0	–	Q(t)
1	0	0
1	1	1

Positive-edge-triggered D Flip-Flop

Graphical symbol



Truth table

Clk	D	Q(t+1)
↑	0	0
↑	1	1
0	–	Q(t)
1	–	Q(t)

```

LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY latch IS
    PORT ( D, Clock : IN      STD_LOGIC ;
           Q          : OUT    STD_LOGIC);
END latch ;

ARCHITECTURE behavioral OF latch IS
BEGIN
    PROCESS ( D, Clock )
    BEGIN
        IF Clock = '1' THEN
            Q <= D ;
        END IF ;
    END PROCESS ;
END behavioral;

```

VHDL Code for a D Flip-Flop with Asynchronous Reset

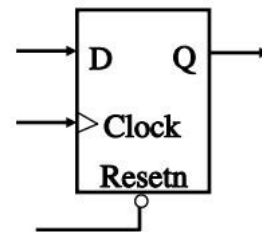
```

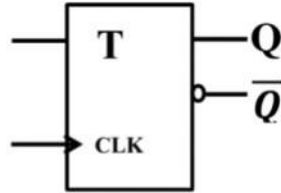
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY flipflop_ar IS
    PORT ( D, Resetn, Clock : IN      STD_LOGIC ;
           Q                : OUT    STD_LOGIC);
END flipflop_ar ;

ARCHITECTURE behavioral OF flipflop_ar IS
BEGIN
    PROCESS ( Resetn, Clock )
    BEGIN
        IF Resetn = '0' THEN
            Q <= '0' ;
        ELSIF rising_edge(Clock) THEN
            Q <= D ;
        END IF ;
    END PROCESS ;
END behavioral ;

```





The truth table of a T-flip-flop

T	Q_{t+1}
0	Q_t
1	$\overline{Q_t}$

VHDL Code for a T Flip Flop

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

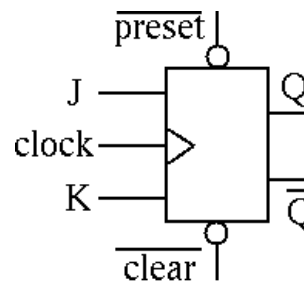
entity Toggle_flip_flop is
    port(
        t : in STD_LOGIC;
        clk : in STD_LOGIC;
        reset : in STD_LOGIC;
        dout : out STD_LOGIC
    );
end Toggle_flip_flop;

architecture virat of Toggle_flip_flop is
begin
    tff : process (t,clk,reset) is
        variable m : std_logic := '0';

    begin
        if (reset = '1') then
            m := '0';
        elsif (rising_edge (clk)) then
            if (t = '1') then
                m := not m;
            end if;
        end if;
        dout <= m;
    end process tff;
end virat;
```

VHDL code for a

JK Flip Flop



J	K	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	$\overline{Q_t}$

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity jk is
  port(
    j : in STD_LOGIC;
    k : in STD_LOGIC;
    clk : in STD_LOGIC;
    reset : in STD_LOGIC;
    q : out STD_LOGIC;
    qb : out STD_LOGIC
  );
end jk;

architecture virat of jk is
begin
  jkff : process (j,k,clk,reset) is
    variable m : std_logic := '0';

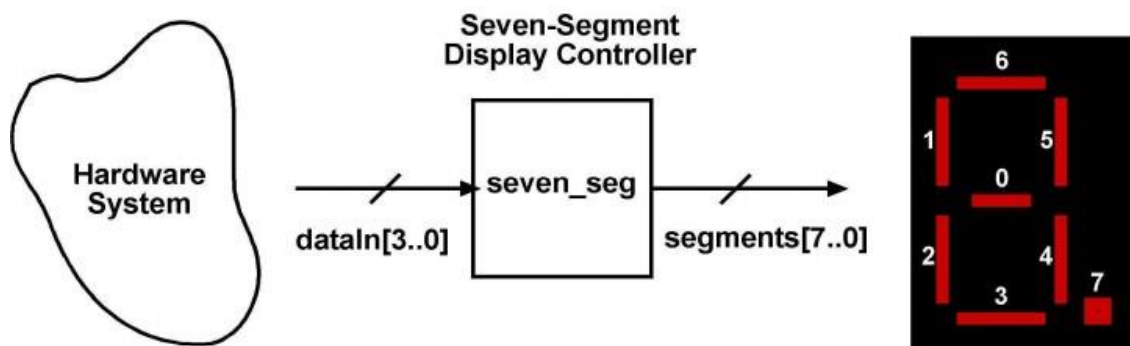
  begin
    if (reset = '1') then
      m := '0';
    elsif (rising_edge (clk)) then
      if (j /= k) then
        m := j;
      elsif (j = '1' and k = '1') then
        m := not m;
      end if;
    end if;

    q <= m;
    qb <= not m;
  end process jkff;
end virat;

```

Design synthesizable VHDL specification of seven segment display controller.

The seven segment display controller is shown below.



```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY seven_seg IS
    PORT( dataIn      : IN      std_logic_vector(3 DOWNTO 0);
          segments    : OUT     std_logic_vector(7 DOWNTO 0) );
END seven_seg;

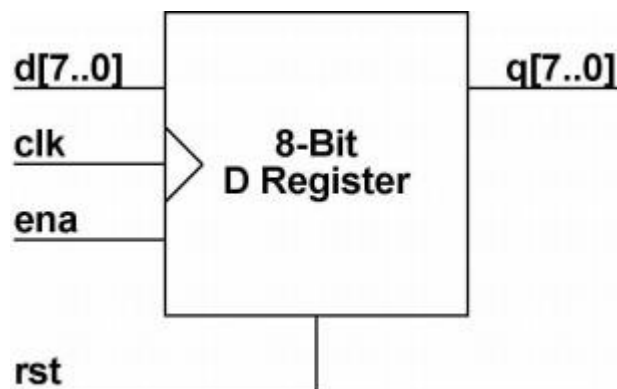
ARCHITECTURE synthesis1 OF seven_seg IS
BEGIN
    WITH dataIn SELECT
        segments <=
            "10000001" WHEN "0000",    -- 0
            "11001111" WHEN "0001",    -- 1
            "10010010" WHEN "0010",    -- 2
            "10000110" WHEN "0011",    -- 3
            "11001100" WHEN "0100",    -- 4
            "10100100" WHEN "0101",    -- 5
            "10100000" WHEN "0110",    -- 6
            "10001111" WHEN "0111",    -- 7
            "10000000" WHEN "1000",    -- 8
            "10000100" WHEN "1001",    -- 9
            "11111111" WHEN OTHERS;

END synthesis1;

```

Design the synthesizable VHDL specification of a 8-bit register with enable and asynchronous reset signal.

The block diagram of 8 –bit register shown below.



```

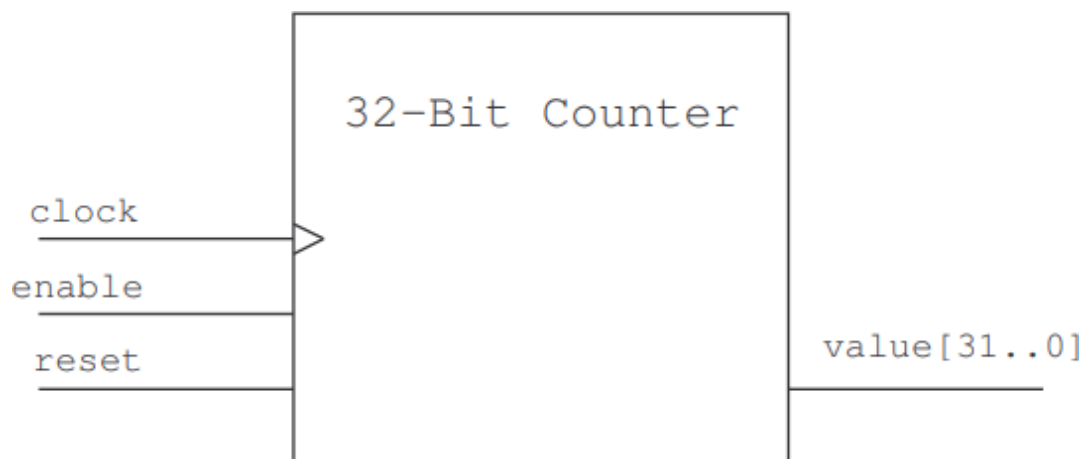
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY dregister IS
    PORT( rst, clk, ena    : IN      std_logic;
          d                : IN      std_logic_vector(7 DOWNTO 0);
          q                : OUT     std_logic_vector(7 DOWNTO 0) );
END dregister;

ARCHITECTURE synthesis1 OF dregister IS
BEGIN
    PROCESS (rst, clk)
    BEGIN
        IF (rst = '1') THEN
            q <= X"00";
        ELSIF (clk'EVENT) AND (clk = '1') THEN
            IF (ena = '1') THEN
                q <= d;
            END IF;
        END IF;
    END PROCESS;
END synthesis1;

```

Design the 32-bit sequence counter using VHDL.



```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY counter IS
    PORT (
        reset          : IN    std_logic;
        clock           : IN    std_logic;
        enable          : IN    std_logic;
        value           : OUT   std_logic_vector(31 DOWNT0 0)
    );
END counter;

```

```

ARCHITECTURE synthesis1 OF counter IS

```

```

    SIGNAL count : unsigned(31 DOWNT0 0); -- The unsigned type is used
                                           -- so that unsigned arithmetic
                                           -- will be synthesized

BEGIN
    PROCESS (reset, clock)
    BEGIN
        IF (reset = '1') THEN
            count <= X"00000000";
        ELSIF (clock'EVENT) AND (clock = '1') THEN
            IF (enable = '1') THEN
                count <= count + 1;
            END IF;
        END IF;
    END PROCESS;

    value <= std_logic_vector(count); -- Here, the count value is
                                      -- converted to std_logic_vector
                                      -- using a conversion function

END synthesis1;

```
